Joint Offloading and Layer Splitting for Resource-Limited Split Federated Learning

Shu Hong, *Member, IEEE,* and Vincent W.S. Wong, *Fellow, IEEE*

***Abstract*—The growing scale of deep models is straining learning at the edge: resource-constrained devices struggle to train locally, and heterogeneous wireless links create communication bottlenecks, resulting in stragglers. To address these challenges, in this paper, we propose a multi-edge split federated learning (MESFL) framework, which facilitates split federated learning (SFL) over a mobile edge computing (MEC) system. The MEC system comprises client devices, edge servers, and a federated (fed) server. Server-side sub-model aggregation is performed on each edge server. The periodic federated aggregation is performed on the fed server. We consider a practical resource-limited scenario where all client devices and edge servers have limited computation and memory capacities. The proposed MESFL framework jointly optimizes sub-model offloading decisions, layer splitting, and federated aggregation frequency to improve the learning efficiency. We formulate a mixed-integer nonlinear programming (MINLP) problem to minimize the learning convergence bound subject to latency and memory constraints. The problem is NP-hard, and we apply an exact finite-interval decomposition with interval-level pruning, which yields a globally optimal solution when the fixed-interval subproblems are solved to optimality, or a certified anytime solution under a time limit. Extensive experiments on the MNIST, CIFAR-10, and CIFAR-100 datasets with ResNet-18, together with an architecture study on CIFAR-10 using VGG-11 and MobileNetV2, validate the effectiveness of the proposed MESFL framework. We compare MESFL with seven baseline methods in terms of test accuracy and training latency. We further conduct an ablation analysis to study the effects of offloading and layer splitting decisions.**

***Index Terms*—Split learning, federated learning, hierarchical offloading, device heterogeneity**

I. Introduction

With the rapid development of Internet of things (IoT) applications, massive amounts of data are generated by different types of devices (e.g., sensors, smartphones). Distributed machine learning (ML) has therefore become increasingly important [1]. Leveraging on-device computation capabilities enables local model training while preserving data privacy by keeping the raw data on the devices.

Federated learning (FL) [2]–[4] is a prominent distributed ML framework, in which each client downloads a global model, updates the model using its own local data, and then transmits the model updates to a central server for aggregation. Despite its wide range of applications, FL faces significant bottlenecks due to the limited computation, communication, and memory capabilities of edge devices. Deep neural networks (DNNs), such as VGG-16 [5] and ResNet-18 [6], require substantial computational power and memory. For instance, ResNet-18, with approximately 11.7 million parameters, requires around 46.8 MB of memory in single-precision floating-point format and a single forward pass requires on the order of a few giga floating point operations (GFLOPs). These requirements impose a heavy burden on resource-constrained devices. Frequent exchange of high-dimensional model parameters also introduces significant communication overhead. Moreover, device heterogeneity (e.g., differences in central processing unit (CPU) speed, memory capacity, and wireless bandwidth) exacerbates these challenges: Some devices can become stragglers that slow down each training round and thus reduce the rate of convergence. These challenges highlight the need for more efficient approaches to support FL in resource-constrained environments.

Recent advances in split learning (SL) [7], [8], which is another type of distributed learning framework, address some of these challenges in FL. SL divides the ML model into the device-side and server-side sub-models at the *split layer*. Each device processes its local data with the device-side sub-model up to the split layer. Each device transmits the resulting intermediate outputs of the split layer to the server. The server continues the processing (forward/backward pass) using its server-side sub-model and transmits the intermediate gradients at the split layer back to the devices, enabling device-side model updates. By offloading the computation-intensive part to a more powerful server, SL mitigates device computation limitations on distributed devices. Communication shifts from the full model updates as in FL to per mini-batch intermediate tensors (i.e., activations and intermediate gradients), which can reduce the communication overhead. Since the raw data in each device are not shared with other devices, SL also preserves privacy. However, conventional SL typically uses sequential training, where each device interacts with the server one at a time. This can lead to long latency and issues such as catastrophic forgetting, where the server-side model continually overfits to the most recent device's data and forgets earlier devices' distributions [9]. Therefore, there is a critical need to improve SL methods in order to fully leverage their potential advantages in resource-constrained environments.

To combine the benefits of both SL and FL, split federated learning (SFL) [9]–[13] and parallel split learning (PSL) [14]–[16] have been proposed as promising approaches for distributed learning. Both frameworks parallelize the device-

This work was supported in part by the Natural Sciences and Engineering Research Council of Canada and the Digital Research Alliance of Canada (alliancecan.ca). Corresponding author: Vincent W.S. Wong

S. Hong and V. W.S. Wong are with the Department of Electrical and Computer Engineering, The University of British Columbia, Vancouver, BC, V6T 1Z4, Canada (e-mail: {shuhong, vincentw}@ece.ubc.ca).

Color versions of one or more of the figures in this paper are available online at https://ieeexplore.ieee.org.

side training up to the split layer across multiple devices and rely on a server for the server-side sub-model training. They can mitigate catastrophic forgetting and improve the training efficiency. The difference between SFL and PSL lies in the device-side sub-model aggregation. In particular, SFL also performs device-side sub-model aggregation at the federated (fed) server. It can provide a higher accuracy at the expense of additional communication overhead due to device-side sub-model exchange. On the other hand, PSL does not aggregate device-side sub-models. Only the server-side sub-model is updated using activations from all the devices. Although PSL can incur a lower communication overhead when compared with SFL, PSL may lead to a lower accuracy. The efficient parallel split learning (EPSL) framework proposed in [15] can accelerate model training by parallelizing client-side training and reducing the dimension of local gradients during backpropagation. It can reduce the per-round latency and communication overhead, especially in resource-constrained wireless edge networks. The hierarchical split federated learning (HSFL) framework proposed in [17] extends SFL to multi-tier systems. The aforementioned works can improve the utilization and mitigate catastrophic forgetting via parallelism and hierarchy.

In both SFL and PSL, the training accuracy and the latency depend on the layer splitting decisions. Specifically, the training accuracy depends on the extent of server-side aggregation, which is influenced by the chosen split layer. A deeper split (i.e., more layers on the device side) leaves fewer layers on the server side. This reduces the portion of the model that benefits from cross-device aggregation. On the other hand, the training latency depends on the computation workload on the devices and servers, as well as the data size of intermediate transmissions. Assigning more layers to the device increases local computation and the size of the activation tensors to be transmitted to the server. This may lead to a longer delay due to stragglers. In contrast, offloading more layers to the server reduces device computation but increases server-side processing time. This can become a bottleneck if the resources at the server side are limited. The existing works on SFL and PSL either fix the split layer [9], [10], [14] or optimize it solely to minimize the per-round training latency [12], [15], [16], [18]. These approaches do not consider the tradeoff between split layer decisions, training accuracy, and latency. The AdaptSFL framework proposed in [19] aims to minimize the training latency subject to a constraint on the convergence bound. However, it does not consider the additional latency due to device heterogeneity.

Although the aforementioned works [9]–[19] are promising, the existing SFL and PSL approaches face several limitations due to their reliance on a single server for split model training. First, the computation load can still be high for a single resource-limited server as the number of devices increases. Even when employing a cloud server, the round-trip time for transmitting the intermediate results to the distant cloud introduces substantial latency in each training round. Second, the latency is constrained by the slowest device. In conventional synchronous training, the server has to wait for all the devices to update their activations before it can proceed with training and perform server-side sub-model aggregation. The diverse communication and computation capabilities among heterogeneous devices exacerbate this issue. Although recent studies [20], [21] have explored the use of multiple edge servers within SL frameworks, they lack a thorough analysis of the latency, layer splitting decisions, and training performance.

In this work, we address these limitations by developing a generalized multi-edge split federated learning (MESFL) framework for a mobile edge computing (MEC) system [22]. Our proposed MESFL framework leverages multiple edge servers for server-side sub-model training and one fed server for periodic model aggregation in resource-limited scenarios. Each edge server has limited computation and memory resources. The edge servers are responsible for parallel server-side sub-model training. This approach can alleviate the need for devices to transmit the intermediate results to a remote cloud in each training round, thereby reducing the communication overhead. Our framework inherently divides the devices into multiple groups based on their offloading decisions to different edge servers. Those devices which belong to the same group offload part of the training workload to the same edge server. In each round, an edge server only needs to wait for devices within its group, which can reduce the latency. It also supports adaptive control of heterogeneous layer splitting (i.e., devices can select different split layers based on their capabilities) and offloading decisions. The server-side sub-models can be efficiently aggregated within each group at the respective edge servers during each splitting round, without incurring extra communication overhead. Model aggregation at the fed server can improve the training accuracy, albeit at the expense of a controllable latency cost. Hence, we consider periodic model aggregation at the fed server, i.e., all entities only exchange the sub-model parameters with the fed server every few rounds. Our proposed MESFL framework can be considered as a generalization of SplitFed [9] and AdaptSFL [19]. The main contributions of this paper are summarized as follows.

- We propose a split federated learning framework called MESFL, with one fed server, multiple edge servers, and a set of client devices. Since the client devices have different computation and memory capacities, we utilize the computation and memory resources of the edge servers for model splitting, while employing a fed server for periodic model aggregation. This setting introduces coupled offloading and model splitting decisions.
- We provide a theoretical analysis of the learning convergence and total latency of the proposed MESFL framework. Based on the analytical results, we formulate a mixed-integer nonlinear programming (MINLP) problem that minimizes the convergence bound subject to the latency and memory constraints. To the best of our knowledge, this is the first work that jointly optimizes the offloading decisions, layer splitting, and federated aggregation frequency in an SFL framework.
- To solve the formulated MINLP problem, which is NP-hard because it contains binary decision variables and nonconvex constraints, we first linearize all products

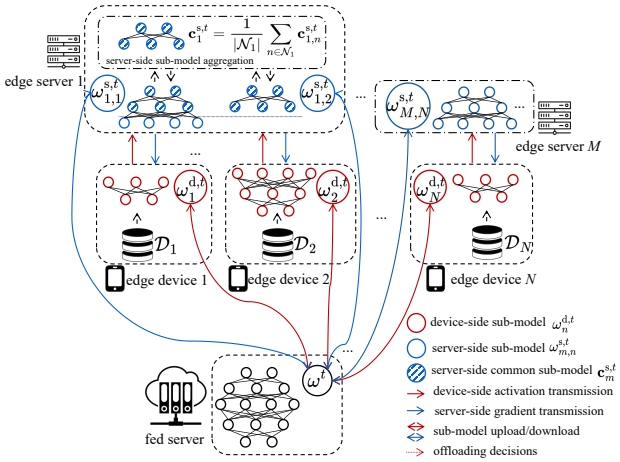


Fig. 1. System model for the proposed MESFL framework. Edge servers perform server-side sub-model aggregation in each round. The fed server performs periodic global aggregation (device- and server-side).

of decision variables. The remaining nonlinearity then lies in the terms involving the aggregation interval E, which is a scalar integer variable with a bounded range. We decompose the formulated problem into multiple compact fixed-interval MILP subproblems, one for each admissible value of E. We derive a monotone interval-level lower bound to prune some subproblems and employ feasible relaxation-based warm starts and group-configuration cuts to speed up finding a solution.

- We conduct experiments on the MNIST, CIFAR-10, and CIFAR-100 datasets with ResNet-18, and further evaluate ResNet-18, VGG-11, and MobileNetV2 on CIFAR-10. We compare our proposed MESFL with FL [2], SL [7], SplitFed [9], AdaptSFL [19], HSFL [17], EPSL [15], and ESFL [18] as baselines. We evaluate the accuracy–latency tradeoff of MESFL and the baseline schemes. We also analyze the impact of various factors including the offloading and layer splitting decisions, showing the performance improvement of joint optimization.

The rest of the paper is organized as follows. In Section II, we introduce the system model and the proposed MESFL framework. In Section III, we present the latency and convergence analysis, as well as the problem formulation. In Section IV, we describe the solution approach to solve the formulated problem. We present the experimental results in Section V. The conclusion is given in Section VI.

II. System Model

In this section, we introduce the system model of the proposed MESFL framework. In contrast to training solely on distant, centralized clouds or on resource-limited devices, our framework leverages edge servers, such as access points (APs) or base stations (BSs) at the wireless network edge. These edge servers mitigate the computational burden on devices and assist in aggregating server-side sub-models. We also employ a fed server for periodic global aggregations, thereby enhancing the overall model convergence.

TABLE I
List of Key Notations

Symbol	Description
$\mathcal{N}$, N	Set and number of devices
$\mathcal{M}$, M	Set and number of edge servers
$\mathcal{N}_m$, N_m	Set and number of devices assigned to edge server m
$\mathcal{L}$, L	Set of candidate split layers $\{1,\dots,L-1\}$ and number of model layers
$\mu_{n,l}$, l_n	Layer splitting binary decision variable for device n and layer l, and the selected split layer of device n
$\boldsymbol{\omega}_n$, $\boldsymbol{\omega}_n^{\mathrm{d}}$, $\boldsymbol{\omega}_{m,n}^{\mathrm{s}}$	Model of device n, its device-side sub-model, and its server-side sub-model offloaded to edge server m
$\mathbf{c}_{m,n}^{\mathrm{s},t}$, $\mathbf{c}_m^{\mathrm{s},t}$, $\mathbf{u}_{m,n}^{\mathrm{s},t}$	Common part of the server-side sub-model of device n, its aggregated value in group m, and the non-common part, in round t
$\beta_{n,m}$	Offloading binary decision variable for device n and edge server m
χ_m, M^+	Active edge server indicator for server m and number of active edge servers
$L_m^\star$	Deepest split layer within group m
E, E^{th}	Aggregation interval and its upper bound from the step-size condition
B, η, T	Batch size, learning rate, number of training rounds
G_k^2, H_l	Second-moment bound of the layer-k stochastic gradient and its cumulative sum over layers 1 to l
Σ_L, ζ^2, S	Total gradient-noise variance, dissimilarity, smoothness
f_n, f_m	Computation capabilities of device n and edge server m
$r_{n,m}^{\mathrm{UL}}$, $r_{m,n}^{\mathrm{DL}}$	Uplink rate from device n to edge server m and downlink rate from edge server m to device n
ψ_l, λ_l	Activation and gradient sizes at split layer l
δ_l, θ_l	Device-side sub-model size and memory footprint for split layer l
ρ_l, α_l	FP and BP workloads of the first l layers
C_n, C_m	Memory capacities of device n and edge server m
$T^{\max}$, J	Latency budget and bound-based surrogate objective

The system model of our proposed framework is illustrated in Fig. 1. It consists of N devices, M edge servers, and one fed server[1]. The devices and edge servers have limited computational and memory resources. Table I summarizes the key notations used throughout the paper.

Let $\mathcal{N}$ and $\mathcal{M}$ denote the sets of devices and edge servers, respectively. Each device $n \in \mathcal{N}$ has a local dataset $\mathcal{D}_n = \{\boldsymbol{x}_i, y_i\}_{i=1}^{|\mathcal{D}_n|}$, where $\boldsymbol{x}_i$ and y_i denote a data sample and its corresponding label, respectively. Let $\ell(\boldsymbol{x}_i, y_i; \boldsymbol{\omega})$ denote the loss for sample $(\boldsymbol{x}_i, y_i)$ under model parameter $\boldsymbol{\omega}$. The local loss function for device n is given by

$$F_n(\boldsymbol{\omega}) = \frac{1}{|\mathcal{D}_n|} \sum_{(\boldsymbol{x}_i, y_i) \in \mathcal{D}_n} \ell(\boldsymbol{x}_i, y_i; \boldsymbol{\omega}). \tag{1}$$

The global learning objective is to minimize the weighted average of the local losses. The global loss function is

$$F(\boldsymbol{\omega}) = \frac{\sum_{n \in \mathcal{N}} |\mathcal{D}_n| F_n(\boldsymbol{\omega})}{\sum_{n' \in \mathcal{N}} |\mathcal{D}_{n'}|}. \tag{2}$$

The optimal global model parameter is given by $\boldsymbol{\omega}^* = \arg\min_{\boldsymbol{\omega}} F(\boldsymbol{\omega})$. For ease of exposition, we assume that all devices have the same number of local samples, namely

[1]We use distinct terms for the edge server and the fed server to differentiate the different roles they play. Although some works may use one entity to act as those two servers, this may lead to privacy risks, considering that both middle-layer sub-models and their outputs would be available at the same server.

$|\mathcal{D}_1| = \cdots = |\mathcal{D}_N|$. Hence the global objective in (2) reduces to

$$F(\boldsymbol{\omega}) = \frac{1}{N} \sum_{n=1}^{N} F_n(\boldsymbol{\omega}). \tag{3}$$

A. MESFL Framework

In our proposed MESFL framework, all devices and servers collaboratively train a machine learning model. Due to computation and memory limitations, each device n partitions its model $\boldsymbol{\omega}_n$ at the split layer. It then offloads the server-side sub-model to one of the edge servers. Specifically, device n's model is split as

$$\boldsymbol{\omega}_n = \big[\underbrace{\boldsymbol{\omega}_n^{\mathrm{d}}}_{\text{layers } 1,\ldots,l_n} , \underbrace{\boldsymbol{\omega}_{m,n}^{\mathrm{s}}}_{\text{layers } l_n+1,\ldots,L} \big] \tag{4}$$

at the split layer $l_n \in \mathcal{L}$, where L denotes the number of layers of the entire model and $\mathcal{L} = \{1, \ldots, L-1\}$ denotes the set of candidate split layers. There are $L-1$ candidate split points, and the server-side sub-model always contains at least the output layer. The device-side sub-model $\boldsymbol{\omega}_n^{\mathrm{d}}$ consists of layers 1 to l_n of the neural network. The server-side sub-model $\boldsymbol{\omega}_{m,n}^{\mathrm{s}}$, if being offloaded to edge server m, consists of layers $l_n + 1$ to L.

Let $\mathcal{N}_m \subseteq \mathcal{N}$ denote the set of devices that offload their server-side sub-models to edge server m. Such sets are determined by the offloading decisions of all devices, which will be presented in Section III. We refer to edge server m and its associated devices $n \in \mathcal{N}_m$ as group m. Within each group, the devices perform device-side sub-model training and transmission in parallel. The corresponding edge server manages the server-side sub-model training and aggregation in each training round.

We define *one training round* $t \in \mathcal{T} = \{1, 2, \ldots, T\}$ of device n as a complete update cycle of model $\boldsymbol{\omega}_n$, including both the forward propagation (FP) and backpropagation (BP) on the device-side and server-side, together with the transmission of intermediate activations or gradients. To ensure model consistency across groups, the fed server aggregates both device-side and server-side sub-models periodically. Specifically, every E rounds, all devices $n \in \mathcal{N}_m$ in group m and their corresponding edge server upload their sub-models to the fed server. After receiving updates from all groups, the fed server performs global aggregation and broadcasts the updated sub-models to all devices and edge servers.

B. Split Model Training with Edge Servers

We now present the split model training within a specific group, including devices that offload their sub-models to the same edge server. We consider T training rounds. With layers $l_n + 1$ to L offloaded to an edge server, the edge server will keep all server-side sub-models and perform the aggregation in each round without incurring extra communication overhead.

1) Device-side Forward Propagation: In this step, all devices perform device-side FP in parallel. Specifically, in training round $t \in \mathcal{T}$, device n randomly samples a batch of data $\mathcal{B}_n^t$ of size B from its local dataset $\mathcal{D}_n$, obtaining inputs $\mathbf{X}_n^t$ and the corresponding labels $\mathbf{Y}_n^t$. Using the device-side sub-model $\boldsymbol{\omega}_n^{\mathrm{d},t}$, device n generates activations at the split layer as $\mathbf{S}_n^t = f(\mathbf{X}_n^t; \boldsymbol{\omega}_n^{\mathrm{d},t})$, where $f(\boldsymbol{x}; \boldsymbol{w})$ is the mapping function from the input data $\boldsymbol{x}$ and model parameter $\boldsymbol{w}$ to the predicted value. Device n then uploads the intermediate activation $\mathbf{S}_n^t$ and the corresponding labels[2] $\mathbf{Y}_n^t$ to an edge server.

2) Server-side Model Forward Propagation: When edge server m has received activations $\{\mathbf{S}_n^t\}_{n \in \mathcal{N}_m}$, it performs server-side FP using $\mathbf{S}_n^t$ as input to the server-side sub-model. The server-side output (prediction) for device n in training round t is given as $\hat{\mathbf{y}}_n^t = f(\mathbf{S}_n^t; \boldsymbol{\omega}_{m,n}^{\mathrm{s},t})$.

However, due to devices' heterogeneous layer splitting decisions, the server-side sub-model aggregation raises a challenge. As different devices offload different numbers of layers, their server-side sub-models can have different depths and thus cannot be averaged directly in the same manner as in federated averaging (FedAvg) [2].

To address this issue, we divide the server-side sub-model for device n at edge server m as

$$\boldsymbol{\omega}_{m,n}^{\mathrm{s},t} = [\mathbf{u}_{m,n}^{\mathrm{s},t}, \mathbf{c}_{m,n}^{\mathrm{s},t}], \tag{5}$$

where $\mathbf{u}_{m,n}^{\mathrm{s},t}$ and $\mathbf{c}_{m,n}^{\mathrm{s},t}$ denote the non-common and common parts of the server-side sub-model of device n, respectively [16], [19]. The common part covers the layers that are held by all devices in group m. It corresponds to the server-side sub-model with the smallest number of layers across $n \in \mathcal{N}_m$, namely the layers from $\max_{n \in \mathcal{N}_m} l_n + 1$ to L. The non-common part covers the remaining layers from $l_n + 1$ to $\max_{n' \in \mathcal{N}_m} l_{n'}$. After the server-side update of round t, each edge server m aggregates the common parts of the devices in its group as

$$\mathbf{c}_m^{\mathrm{s},t+1} = \frac{1}{|\mathcal{N}_m|} \sum_{n \in \mathcal{N}_m} \mathbf{c}_{m,n}^{\mathrm{s},t+1}, \tag{6}$$

and then sets $\mathbf{c}_{m,n}^{\mathrm{s},t+1} := \mathbf{c}_m^{\mathrm{s},t+1}$ for all $n \in \mathcal{N}_m$, so that the common part is identical across the group at the beginning of round $t+1$. The non-common sub-model arises from devices' different layer splitting decisions l_n, which are periodically aggregated at the fed server along with the device-side sub-models.

3) Server-side Model Backpropagation: In this step, the predicted value $\hat{\mathbf{y}}_n^t$ and the corresponding ground-truth labels $\mathbf{Y}_n^t$ are used to determine the loss function. The gradients of the loss function with respect to the server-side sub-model parameters $\boldsymbol{\omega}_{m,n}^{\mathrm{s},t}$ are then obtained. Using the stochastic gradient descent (SGD) approach, the server-side sub-model parameters are updated as follows:

$$\boldsymbol{\omega}_{m,n}^{\mathrm{s},t+1} = \boldsymbol{\omega}_{m,n}^{\mathrm{s},t} - \eta \mathbf{g}_n^{\mathrm{s},t}, \tag{7}$$

where η is the learning rate and $\mathbf{g}_n^{\mathrm{s},t}$ is the mini-batch gradient of the local loss function F_n with respect to the server-side

[2]Potential privacy risks associated with label transmission can be mitigated by shifting the final layer's loss calculation back to the device-side [7], [21], [23]. Specifically, after the edge servers have completed forward propagation, the activations from the penultimate layer are transmitted back to the corresponding devices. The devices, which hold the labels, calculate the loss function and send the resulting gradients back to the edge servers. This variant adds one downlink and one uplink transmission per round, and its overhead is quantified in Section III-B. In this paper, the labels are transmitted to the edge servers.

sub-model parameters[3]. The gradients are computed sequentially from the output layer down to the split layer via the chain rule. Once the gradient calculation has proceeded to the split layer, the gradient is sent back to the corresponding device.

4) Device-side Model Backpropagation: In this step, each device n updates its device-side sub-model parameters based on the received activations' gradients:

$$\boldsymbol{\omega}_n^{\mathrm{d},t+1} = \boldsymbol{\omega}_n^{\mathrm{d},t} - \eta \mathbf{g}_n^{\mathrm{d},t}, \tag{8}$$

where $\mathbf{g}_n^{\mathrm{d},t}$ is the mini-batch gradient of F_n with respect to the device-side sub-model parameters, computed by backpropagating the split-layer gradient returned by the corresponding edge server through the device-side sub-model. The concatenation $[\mathbf{g}_n^{\mathrm{d},t}, \mathbf{g}_n^{\mathrm{s},t}]$ is the mini-batch gradient of F_n with respect to the full model, which is denoted by $\mathbf{g}_n$ in Section III.

C. Periodic Federated Aggregation

Unlike server-side aggregation (which occurs every round at each edge server), sending sub-models from the devices and edge servers to a fed server incurs additional communication overhead. Hence, we consider periodic model aggregation at the fed server to balance the trade-off between model consistency and the communication overhead.

We perform global aggregation every E training rounds. That is, the number of local training rounds between two consecutive federated synchronizations is equal to E. We let $t = \tau E$, where $\tau \in \{1, 2, \ldots, \lfloor T/E \rfloor\}$. At the end of round t (i.e., after completing E rounds of split training), all devices and the corresponding edge servers upload their device-side and server-side sub-models to the fed server, respectively. After receiving the sub-model updates from all devices and edge servers, the fed server updates the global parameter using the FedAvg algorithm [2] as

$$\boldsymbol{\omega}^t = \frac{1}{N} \sum_{n=1}^{N} \boldsymbol{\omega}_n^t, \tag{9}$$

where $\boldsymbol{\omega}^t$ represents the concatenation of both device- and server-side sub-models as in (4). The updated global parameters are sent to all devices and edge servers via broadcast prior to round $t+1$. The federated aggregation interval E is a critical decision variable that determines the frequency of synchronization, balancing the communication overhead and convergence speed.

Algorithm 1 summarizes the MESFL framework under a given configuration, i.e., the split layers $\{l_n\}$, the device groups $\{\mathcal{N}_m\}$, and the aggregation interval E. The framework balances the computational efficiency and model convergence. Specifically, periodic aggregation at the fed server ensures global model consistency across all devices and edge servers. The edge servers perform server-side sub-model aggregation in each round to reduce the communication overhead. The joint optimization of the configuration is formulated in Section III and solved by Algorithm 2 in Section IV. This framework generalizes several existing SFL methodologies, offering improved scalability and efficiency. When $M = 1$ and $E = 1$ with homogeneous layer splitting, it is equivalent to SplitFed in [9]. In this case, all device-side sub-models are aggregated at the fed server after each round. This can lead to a high latency due to devices' heterogeneous capabilities and the large size of sub-model parameters. When $M = 1$ and $E > 1$, our framework aligns with AdaptSFL in [19]. In this case, a single

[3]Here, $\mathbf{g}_n^{\mathrm{s},t}$ is computed from the average loss between the predictions $\hat{\mathbf{y}}_n^t$ and the labels $\mathbf{Y}_n^t$ over the mini-batch $\mathcal{B}_n^t$, which is an unbiased estimate of the corresponding block of ∇F_n.

Algorithm 1 Multi-Edge Split Federated Learning (MESFL) Framework

Input: Number of training rounds T, aggregation interval E, learning rate η, split layers $\{l_n\}_{n\in\mathcal{N}}$, and device groups $\{\mathcal{N}_m\}_{m\in\mathcal{M}}$
Initialize: Global model $\boldsymbol{\omega}^0$, split into $\{\boldsymbol{\omega}_n^{\mathrm{d},0}\}$ and $\{\boldsymbol{\omega}_{m,n}^{\mathrm{s},0}\}$ at the split layers $\{l_n\}$ according to (4)
for each round $t = 0, 1, \ldots, T-1$ **do**
  for each device $n \in \mathcal{N}$ **in parallel do**
    Sample mini-batch $\mathcal{B}_n^t := \{(\mathbf{X}_n^t, \mathbf{Y}_n^t)\} \subseteq \mathcal{D}_n$ of size B
    Compute activation at split layer: $\mathbf{S}_n^t := f(\mathbf{X}_n^t; \boldsymbol{\omega}_n^{\mathrm{d},t})$
    Upload $\mathbf{S}_n^t$ and $\mathbf{Y}_n^t$ to its edge server m (i.e., $n \in \mathcal{N}_m$)
  end for
  for each edge server $m \in \mathcal{M}$ **do**
    if $\mathcal{N}_m \neq \emptyset$ **then**
      for each device $n \in \mathcal{N}_m$ **do**
        Compute prediction: $\hat{\mathbf{y}}_n^t := f(\mathbf{S}_n^t; \boldsymbol{\omega}_{m,n}^{\mathrm{s},t})$
        Compute the mini-batch loss and the server-side gradient $\mathbf{g}_n^{\mathrm{s},t}$
        Update server-side sub-model: $\boldsymbol{\omega}_{m,n}^{\mathrm{s},t+1} := \boldsymbol{\omega}_{m,n}^{\mathrm{s},t} - \eta \mathbf{g}_n^{\mathrm{s},t}$
        Send the split-layer gradient to device n
        Obtain $\mathbf{c}_{m,n}^{\mathrm{s},t+1}$ from updated $\boldsymbol{\omega}_{m,n}^{\mathrm{s},t+1}$
      end for
      Aggregate common sub-model: $\mathbf{c}_m^{\mathrm{s},t+1} := \frac{1}{|\mathcal{N}_m|} \sum_{n\in\mathcal{N}_m} \mathbf{c}_{m,n}^{\mathrm{s},t+1}$
      Set $\mathbf{c}_{m,n}^{\mathrm{s},t+1} := \mathbf{c}_m^{\mathrm{s},t+1}$ in $\boldsymbol{\omega}_{m,n}^{\mathrm{s},t+1}$ for all $n \in \mathcal{N}_m$
    end if
  end for
  for each device $n \in \mathcal{N}$ **in parallel do**
    Compute $\mathbf{g}_n^{\mathrm{d},t}$ via backpropagation from the split-layer gradient
    Update device-side sub-model: $\boldsymbol{\omega}_n^{\mathrm{d},t+1} := \boldsymbol{\omega}_n^{\mathrm{d},t} - \eta \mathbf{g}_n^{\mathrm{d},t}$
  end for
  if $(t+1) \bmod E = 0$ **then**
    Devices upload $\boldsymbol{\omega}_n^{\mathrm{d},t+1}$ and edge servers upload $\boldsymbol{\omega}_{m,n}^{\mathrm{s},t+1}$ to the fed server
    Fed server aggregates: $\boldsymbol{\omega}^{t+1} := \frac{1}{N} \sum_{n=1}^{N} \boldsymbol{\omega}_n^{t+1}$
    Fed server broadcasts the updated model to all devices and edge servers
  end if
end for
Output: Trained global model $\boldsymbol{\omega}^T$

edge server has to wait for all devices to upload their sub-model outputs in each round. Faster devices may remain idle waiting for slower devices (stragglers), potentially leading to inefficient use of resources. Both SplitFed and AdaptSFL serve as baselines for evaluating the proposed framework in Section V.

III. Theoretical Analysis and Problem Formulation

In this section, we first analyze the convergence performance and latency of the proposed MESFL framework. We then formulate the joint optimization problem of layer splitting, offloading, and federated aggregation frequency. The objective is to minimize a convergence-bound surrogate subject to latency and memory constraints.

We introduce two sets of binary decision variables. Let $\mu_{n,l} \in \{0,1\}$ denote the layer splitting decision for device $n \in \mathcal{N}$ and layer $l \in \mathcal{L}$. Here, $\mu_{n,l} = 1$ indicates that the l-th layer is selected as the split layer for device n. We have the following constraint

$$\sum_{l \in \mathcal{L}} \mu_{n,l} = 1, \qquad n \in \mathcal{N}, \tag{10}$$

so that each device has one split layer. The first layer is always computed at the device side and hence preserves privacy. The output layer always remains at the edge server since $L \notin \mathcal{L}$. Let $\beta_{n,m} \in \{0,1\}$ denote the offloading decision for device $n \in \mathcal{N}$ and edge server $m \in \mathcal{M}$. The condition $\beta_{n,m} = 1$ indicates that device n offloads its server-side sub-model to edge server m. We have the following constraint

$$\sum_{m=1}^{M} \beta_{n,m} = 1, \qquad n \in \mathcal{N}, \tag{11}$$

so that each device selects one edge server. We define the set of devices that are served by edge server m as

$$\mathcal{N}_m \triangleq \{n \in \mathcal{N} \mid \beta_{n,m} = 1\}, \qquad N_m \triangleq |\mathcal{N}_m|. \tag{12}$$

For compact notation, we collect the decision variables as $\boldsymbol{\mu} \triangleq \{\mu_{n,l}\}_{n \in \mathcal{N}, l \in \mathcal{L}}$ and $\boldsymbol{\beta} \triangleq \{\beta_{n,m}\}_{n \in \mathcal{N}, m \in \mathcal{M}}$. We aim to optimize the binary decision variables $\boldsymbol{\mu}$ and $\boldsymbol{\beta}$, and the federated aggregation interval E. These variables affect both the latency expressions in Section III-B and the convergence bound in Theorem 1 (to be presented in the following subsection).

A. Convergence Analysis

In this subsection, we analyze the convergence performance of our proposed MESFL framework. First we present some assumptions for convergence analysis.

For device n, let ξ_n denote the mini-batch $\mathcal{B}_n^t$ of Section II, with the round index omitted. Then

$$\mathbf{g}_n(\boldsymbol{\omega}, \xi_n) \triangleq \frac{1}{B} \sum_{(\boldsymbol{x}_i, y_i) \in \xi_n} \nabla \ell(\boldsymbol{x}_i, y_i; \boldsymbol{\omega}) \tag{13}$$

denotes the resulting stochastic gradient of F_n at $\boldsymbol{\omega}$. Partitioning the parameter vector into layer blocks as $\boldsymbol{\omega} = [\boldsymbol{\omega}_{(1)}, \ldots, \boldsymbol{\omega}_{(L)}]$, we let $\mathbf{g}_{n,k}(\boldsymbol{\omega}, \xi_n)$ denote the sub-vector of $\mathbf{g}_n(\boldsymbol{\omega}, \xi_n)$ associated with the parameters of layer k. We have $\|\mathbf{g}_n(\boldsymbol{\omega}, \xi_n)\|^2 = \sum_{k=1}^{L} \|\mathbf{g}_{n,k}(\boldsymbol{\omega}, \xi_n)\|^2$.

Assumption 1 (Smoothness and lower boundedness)**.** *Each local loss function $F_n(\boldsymbol{\omega})$ is S-smooth, i.e.,*

$$\|\nabla F_n(\boldsymbol{\omega}) - \nabla F_n(\boldsymbol{\omega}')\| \le S\|\boldsymbol{\omega} - \boldsymbol{\omega}'\|, \qquad \forall \boldsymbol{\omega}, \boldsymbol{\omega}'. \tag{14}$$

The global objective is lower bounded. That is,

$$F^\star = \inf_{\boldsymbol{\omega}} F(\boldsymbol{\omega}) > -\infty. \tag{15}$$

Assumption 2 (Conditional unbiasedness and independent noise)**.** *Let $\mathcal{F}^{t-1}$ denote the sigma-field generated by all randomness before round t. The effective models $\{\tilde{\boldsymbol{\omega}}_n^t\}_{n=1}^N$ are $\mathcal{F}^{t-1}$-measurable. Conditional on $\mathcal{F}^{t-1}$, the mini-batch samples in round t are drawn independently across devices. For each device n, the stochastic gradient of local loss function F_n is conditionally unbiased,*

$$\mathbb{E}[\mathbf{g}_n(\tilde{\boldsymbol{\omega}}_n^t, \xi_n^t) \mid \mathcal{F}^{t-1}] = \nabla F_n(\tilde{\boldsymbol{\omega}}_n^t), \tag{16}$$

and has bounded conditional variance,

$$\mathbb{E}\big[\|\mathbf{g}_n(\tilde{\boldsymbol{\omega}}_n^t, \xi_n^t) - \nabla F_n(\tilde{\boldsymbol{\omega}}_n^t)\|^2 \mid \mathcal{F}^{t-1}\big] \le \Sigma_L. \tag{17}$$

Here the constant Σ_L bounds the total stochastic-gradient noise variance accumulated over all L layers of the model. The resulting stochastic-gradient noises are martingale differences and are conditionally independent across devices and across rounds.

Assumption 3 (Layer-wise second moments)**.** *For each device n and each layer k,*

$$\mathbb{E}\|\mathbf{g}_{n,k}(\boldsymbol{\omega}, \xi_n)\|^2 \le G_k^2, \qquad \forall \boldsymbol{\omega}. \tag{18}$$

Define $H_l = \sum_{k=1}^{l} G_k^2$ and $H_0 = 0$.

Assumption 4 (Inter-client dissimilarity)**.** *There exists a constant $\zeta \ge 0$ such that*

$$\frac{1}{N} \sum_{n=1}^{N} \|\nabla F_n(\boldsymbol{\omega}) - \nabla F(\boldsymbol{\omega})\|^2 \le \zeta^2, \qquad \forall \boldsymbol{\omega}. \tag{19}$$

The special case $\zeta = 0$ corresponds to homogeneous local objectives.

For each device $n \in \mathcal{N}$, its selected split layer is $l_n = \sum_{l \in \mathcal{L}} l \mu_{n,l}$. Let

$$\mathcal{M}^+ = \{m \in \mathcal{M} \mid N_m > 0\}, \qquad M^+ = |\mathcal{M}^+|. \tag{20}$$

For each active edge server $m \in \mathcal{M}^+$, the deepest split layer among its associated devices is $L_m^\star = \max_{n \in \mathcal{N}_m} l_n$. The drift function that depends on the layer splitting and offloading decisions is

$$\Theta(\boldsymbol{\mu}, \boldsymbol{\beta}) = \sum_{m \in \mathcal{M}^+} \frac{N_m}{N} H_{L_m^\star}. \tag{21}$$

We use the convention that all devices share the same L-layer architecture with identical per-layer dimensions and differ only in their split indices. For device n assigned to edge server m, the effective full model $\tilde{\boldsymbol{\omega}}_n^t$ is the concatenation of its device-side layers and the server-side layers that it used in

round t. Thus, every $\tilde{\boldsymbol{\omega}}_n^t$ lies in the same parameter space. Immediately before round t, we define the virtual average model as

$$\bar{\boldsymbol{\omega}}^{t-1} = \frac{1}{N}\sum_{n=1}^{N} \tilde{\boldsymbol{\omega}}_n^t. \tag{22}$$

The virtual average model is used only for analysis and need not coincide with every local effective model between two consecutive fed-server aggregations.

Theorem 1 (MESFL convergence bound)**.** *Under Assumptions 1–4, if $\eta \le \frac{1}{4S}$ and $\eta S(E-1) \le \frac{1}{4}$, then the virtual average sequence $\{\bar{\boldsymbol{\omega}}^t\}$ generated by MESFL satisfies*

$$\begin{aligned}
&\frac{1}{T}\sum_{t=1}^{T} \mathbb{E}\|\nabla F(\bar{\boldsymbol{\omega}}^{t-1})\|^2 \\
&\le \underbrace{\frac{4(F(\bar{\boldsymbol{\omega}}^0) - F^\star)}{\eta T}}_{\textit{initial optimality gap}} + \underbrace{\frac{2S\eta\Sigma_L}{N}}_{\textit{gradient noise}} \\
&\quad + \underbrace{6S^2\eta^2(E-1)^2\Theta(\boldsymbol{\mu},\boldsymbol{\beta})}_{\textit{splitting and offloading drift}} \\
&\quad + \underbrace{48S^2\eta^2(E-1)^2\zeta^2}_{\textit{data heterogeneity drift}} + \underbrace{12S^2\eta^2(E-1)\frac{M^+}{N}\Sigma_L}_{\textit{between-edge noise}}.
\end{aligned} \tag{23}$$

Proof: Throughout the proof, we write $r = E-1$ for brevity. By the update rule, both edge-server common-layer averaging and fed-server aggregation are unweighted arithmetic means over the participating devices, and therefore leave the network-wide average of the corresponding layer blocks unchanged. Hence, the virtual average satisfies

$$\bar{\boldsymbol{\omega}}^t = \bar{\boldsymbol{\omega}}^{t-1} - \eta\bar{\mathbf{g}}^t, \qquad \bar{\mathbf{g}}^t = \frac{1}{N}\sum_{n=1}^{N} \mathbf{g}_n(\tilde{\boldsymbol{\omega}}_n^t, \xi_n^t). \tag{24}$$

Define

$$\bar{\mathbf{h}}^t = \mathbb{E}[\bar{\mathbf{g}}^t \mid \mathcal{F}^{t-1}] = \frac{1}{N}\sum_{n=1}^{N} \nabla F_n(\tilde{\boldsymbol{\omega}}_n^t) \tag{25}$$

and $D^t = \|\nabla F(\bar{\boldsymbol{\omega}}^{t-1}) - \bar{\mathbf{h}}^t\|^2$. By Assumption 1, the global objective F is also S-smooth. Therefore,

$$\begin{aligned}
&\mathbb{E}[F(\bar{\boldsymbol{\omega}}^t) \mid \mathcal{F}^{t-1}] \\
&\le F(\bar{\boldsymbol{\omega}}^{t-1}) - \eta\langle \nabla F(\bar{\boldsymbol{\omega}}^{t-1}), \bar{\mathbf{h}}^t\rangle + \frac{S\eta^2}{2}\mathbb{E}[\|\bar{\mathbf{g}}^t\|^2 \mid \mathcal{F}^{t-1}].
\end{aligned} \tag{26}$$

Let $\mathbf{a}^t = \nabla F(\bar{\boldsymbol{\omega}}^{t-1})$. Young's inequality gives

$$-\langle \mathbf{a}^t, \bar{\mathbf{h}}^t\rangle \le -\frac{1}{2}\|\mathbf{a}^t\|^2 + \frac{1}{2}D^t. \tag{27}$$

Moreover, $\|\bar{\mathbf{h}}^t\|^2 \le 2\|\mathbf{a}^t\|^2 + 2D^t$. By Assumption 2,

$$\begin{aligned}
&\mathbb{E}[\|\bar{\mathbf{g}}^t - \bar{\mathbf{h}}^t\|^2 \mid \mathcal{F}^{t-1}] \\
&= \frac{1}{N^2}\sum_{n=1}^{N} \mathbb{E}[\|\mathbf{g}_n(\tilde{\boldsymbol{\omega}}_n^t, \xi_n^t) - \nabla F_n(\tilde{\boldsymbol{\omega}}_n^t)\|^2 \mid \mathcal{F}^{t-1}] \\
&\le \frac{\Sigma_L}{N}.
\end{aligned} \tag{28}$$

Hence, for $\eta \le 1/(4S)$,

$$\begin{aligned}
\mathbb{E}[F(\bar{\boldsymbol{\omega}}^t)] &\le \mathbb{E}[F(\bar{\boldsymbol{\omega}}^{t-1})] - \frac{\eta}{4}\mathbb{E}\|\nabla F(\bar{\boldsymbol{\omega}}^{t-1})\|^2 \\
&\quad + \frac{3\eta}{4}\mathbb{E}[D^t] + \frac{S\eta^2\Sigma_L}{2N}.
\end{aligned} \tag{29}$$

We next bound D^t. Since $\nabla F(\bar{\boldsymbol{\omega}}^{t-1}) = \frac{1}{N}\sum_{n=1}^{N}\nabla F_n(\bar{\boldsymbol{\omega}}^{t-1})$, Jensen's inequality and smoothness imply

$$\begin{aligned}
D^t &= \left\| \frac{1}{N}\sum_{n=1}^{N} \left(\nabla F_n(\bar{\boldsymbol{\omega}}^{t-1}) - \nabla F_n(\tilde{\boldsymbol{\omega}}_n^t)\right)\right\|^2 \\
&\le \frac{S^2}{N}\sum_{n=1}^{N} \|\bar{\boldsymbol{\omega}}^{t-1} - \tilde{\boldsymbol{\omega}}_n^t\|^2.
\end{aligned} \tag{30}$$

For $m \in \mathcal{M}^+$, define the edge mean

$$\boldsymbol{\omega}_m^{t-1} = \frac{1}{N_m}\sum_{n\in\mathcal{N}_m} \tilde{\boldsymbol{\omega}}_n^t. \tag{31}$$

Since $\bar{\boldsymbol{\omega}}^{t-1} = \sum_{m\in\mathcal{M}^+} \frac{N_m}{N}\boldsymbol{\omega}_m^{t-1}$, the variance decomposition gives

$$\Phi^t \triangleq \frac{1}{N}\sum_{n=1}^{N} \|\bar{\boldsymbol{\omega}}^{t-1} - \tilde{\boldsymbol{\omega}}_n^t\|^2 = \Phi_W^t + \Phi_C^t, \tag{32}$$

where

$$\Phi_W^t = \frac{1}{N}\sum_{m\in\mathcal{M}^+}\sum_{n\in\mathcal{N}_m} \|\boldsymbol{\omega}_m^{t-1} - \tilde{\boldsymbol{\omega}}_n^t\|^2 \tag{33}$$

is the within-edge drift and

$$\Phi_C^t = \sum_{m\in\mathcal{M}^+} \frac{N_m}{N}\|\boldsymbol{\omega}_m^{t-1} - \bar{\boldsymbol{\omega}}^{t-1}\|^2 \tag{34}$$

is the between-edge drift.

We first bound Φ_W^t. Let τ be the most recent global synchronization time before round t. At most $r = E-1$ local updates have occurred since τ. Let $\boldsymbol{\omega}^\tau$ denote the common model held by all devices immediately after that synchronization, and let $\mathcal{I}_t \triangleq \{\tau+1, \dots, t-1\}$ denote the set of intervening local rounds, so that $|\mathcal{I}_t| \le r$. Throughout this proof, the superscript j indexes the local rounds in $\mathcal{I}_t$. For edge server m, let Π_m be the projection onto layers $1, \dots, L_m^\star$. The per-round common-layer averaging at the edge server ensures that the common layers $L_m^\star + 1, \dots, L$ are identical across all devices in group m at the gradient-evaluation point. Hence, these layers do not contribute to within-edge dispersion. The arithmetic mean minimizes the sum of squared distances. Thus,

$$\begin{aligned}
&\frac{1}{N_m}\sum_{n\in\mathcal{N}_m} \|\Pi_m\boldsymbol{\omega}_m^{t-1} - \Pi_m\tilde{\boldsymbol{\omega}}_n^t\|^2 \\
&\le \frac{1}{N_m}\sum_{n\in\mathcal{N}_m} \|\Pi_m\boldsymbol{\omega}^\tau - \Pi_m\tilde{\boldsymbol{\omega}}_n^t\|^2.
\end{aligned} \tag{35}$$

For each $n \in \mathcal{N}_m$,

$$\Pi_m(\boldsymbol{\omega}^\tau - \tilde{\boldsymbol{\omega}}_n^t) = \eta\sum_{j\in\mathcal{I}_t} \Pi_m\mathbf{g}_n^j, \qquad |\mathcal{I}_t| \le r, \tag{36}$$

where $\mathbf{g}_n^j \triangleq \mathbf{g}_n(\tilde{\boldsymbol{\omega}}_n^j, \xi_n^j)$ is the stochastic gradient of device n at local round j. Cauchy's inequality and Assumption 3 give $\mathbb{E}\|\Pi_m \boldsymbol{\omega}^\tau - \Pi_m \tilde{\boldsymbol{\omega}}_n^t\|^2 \le \eta^2 r^2 H_{L_m^\star}$. Thus,

$$\mathbb{E}[\Phi_W^t] \le \eta^2 r^2 \sum_{m\in\mathcal{M}^+} \frac{N_m}{N} H_{L_m^\star} = \eta^2 r^2 \Theta(\boldsymbol{\mu}, \boldsymbol{\beta}). \tag{37}$$

We now bound Φ_C^t. If $r = 0$ (i.e., $E = 1$), then all devices are globally synchronized every round and $\Phi_C^t = 0$. Consider $r \ge 1$. The pairwise variance identity gives

$$\Phi_C^t = \frac{1}{2} \sum_{m,m'\in\mathcal{M}^+} \frac{N_m N_{m'}}{N^2} \|\boldsymbol{\omega}_m^{t-1} - \boldsymbol{\omega}_{m'}^{t-1}\|^2. \tag{38}$$

For $j \in \mathcal{I}_t$, the edge mean follows

$$\boldsymbol{\omega}_m^j = \boldsymbol{\omega}_m^{j-1} - \eta \bar{\mathbf{g}}_m^j, \qquad \bar{\mathbf{g}}_m^j = \frac{1}{N_m} \sum_{n\in\mathcal{N}_m} \mathbf{g}_n(\tilde{\boldsymbol{\omega}}_n^j, \xi_n^j). \tag{39}$$

We express $\bar{\mathbf{g}}_m^j = \bar{\mathbf{h}}_m^j + \boldsymbol{\xi}_m^j$, where

$$\bar{\mathbf{h}}_m^j \triangleq \frac{1}{N_m} \sum_{n\in\mathcal{N}_m} \nabla F_n(\tilde{\boldsymbol{\omega}}_n^j) = \mathbb{E}[\bar{\mathbf{g}}_m^j \mid \mathcal{F}^{j-1}] \tag{40}$$

is the edge-level mean gradient (the group-m counterpart of $\bar{\mathbf{h}}^t$), and $\boldsymbol{\xi}_m^j \triangleq \bar{\mathbf{g}}_m^j - \bar{\mathbf{h}}_m^j$ is the corresponding conditionally zero-mean noise. The edge means are equal at the beginning of the synchronization window. Therefore,

$$\boldsymbol{\omega}_m^{t-1} - \boldsymbol{\omega}_{m'}^{t-1} = -\eta \sum_{j\in\mathcal{I}_t} \left[(\bar{\mathbf{h}}_m^j - \bar{\mathbf{h}}_{m'}^j) + (\boldsymbol{\xi}_m^j - \boldsymbol{\xi}_{m'}^j) \right]. \tag{41}$$

Using $\|\mathbf{x}+\mathbf{y}\|^2 \le 2\|\mathbf{x}\|^2 + 2\|\mathbf{y}\|^2$, we obtain

$$\begin{aligned} \mathbb{E}[\Phi_C^t] \le \eta^2 \sum_{m,m'\in\mathcal{M}^+} \frac{N_m N_{m'}}{N^2} \mathbb{E}\left\| \sum_{j\in\mathcal{I}_t} (\bar{\mathbf{h}}_m^j - \bar{\mathbf{h}}_{m'}^j) \right\|^2 \\ + \eta^2 \sum_{m,m'\in\mathcal{M}^+} \frac{N_m N_{m'}}{N^2} \mathbb{E}\left\| \sum_{j\in\mathcal{I}_t} (\boldsymbol{\xi}_m^j - \boldsymbol{\xi}_{m'}^j) \right\|^2. \end{aligned} \tag{42}$$

For the noise part, the martingale-difference property and conditional independence imply $\mathbb{E}\|\boldsymbol{\xi}_m^j\|^2 \le \Sigma_L / N_m$. Hence,

$$\mathbb{E}\left\| \sum_{j\in\mathcal{I}_t} (\boldsymbol{\xi}_m^j - \boldsymbol{\xi}_{m'}^j) \right\|^2 \le r\Sigma_L \left(\frac{1}{N_m} + \frac{1}{N_{m'}} \right). \tag{43}$$

After summing over pairs,

$$\begin{aligned} \sum_{m,m'\in\mathcal{M}^+} \frac{N_m N_{m'}}{N^2} \mathbb{E}\left\| \sum_{j\in\mathcal{I}_t} (\boldsymbol{\xi}_m^j - \boldsymbol{\xi}_{m'}^j) \right\|^2 \\ \le 2r \frac{M^+}{N} \Sigma_L. \end{aligned} \tag{44}$$

For the mean part, define $\mathbf{d}_m^j = \bar{\mathbf{h}}_m^j - \nabla F(\bar{\boldsymbol{\omega}}^{j-1})$. By adding and subtracting $N_m^{-1} \sum_{n\in\mathcal{N}_m} \nabla F_n(\bar{\boldsymbol{\omega}}^{j-1})$, then using Jensen's inequality, smoothness, and Assumption 4,

$$\sum_{m\in\mathcal{M}^+} \frac{N_m}{N} \mathbb{E}\|\mathbf{d}_m^j\|^2 \le 2S^2 \mathbb{E}[\Phi^j] + 2\zeta^2. \tag{45}$$

Since $\bar{\mathbf{h}}_m^j - \bar{\mathbf{h}}_{m'}^j = \mathbf{d}_m^j - \mathbf{d}_{m'}^j$,

$$\begin{aligned} \sum_{m,m'\in\mathcal{M}^+} \frac{N_m N_{m'}}{N^2} \mathbb{E}\|\bar{\mathbf{h}}_m^j - \bar{\mathbf{h}}_{m'}^j\|^2 \\ \le 8S^2 \mathbb{E}[\Phi^j] + 8\zeta^2. \end{aligned} \tag{46}$$

Cauchy's inequality over the at most r local rounds gives

$$\begin{aligned} \sum_{m,m'\in\mathcal{M}^+} \frac{N_m N_{m'}}{N^2} \mathbb{E}\left\| \sum_{j\in\mathcal{I}_t} (\bar{\mathbf{h}}_m^j - \bar{\mathbf{h}}_{m'}^j) \right\|^2 \\ \le 8rS^2 \sum_{j\in\mathcal{I}_t} \mathbb{E}[\Phi^j] + 8r^2\zeta^2. \end{aligned} \tag{47}$$

Combining the mean and noise bounds,

$$\mathbb{E}[\Phi_C^t] \le 8\eta^2 r S^2 \sum_{j\in\mathcal{I}_t} \mathbb{E}[\Phi^j] + 8\eta^2 r^2 \zeta^2 + 2\eta^2 r \frac{M^+}{N} \Sigma_L. \tag{48}$$

Note that $\mathbb{E}[\Phi^j] = \mathbb{E}[\Phi_W^j] + \mathbb{E}[\Phi_C^j] \le \eta^2 r^2 \Theta + \mathbb{E}[\Phi_C^j]$. Let $R = \max_{j\in\mathcal{I}_t\cup\{t\}} \mathbb{E}[\Phi_C^j]$. From (48),

$$R \le 8\eta^2 r^2 S^2 R + 8\eta^4 r^4 S^2 \Theta + 8\eta^2 r^2 \zeta^2 + 2\eta^2 r \frac{M^+}{N} \Sigma_L. \tag{49}$$

The condition $\eta S r \le 1/4$ gives $8\eta^2 r^2 S^2 \le 1/2$. Hence,

$$\mathbb{E}[\Phi_C^t] \le 16\eta^4 r^4 S^2 \Theta + 16\eta^2 r^2 \zeta^2 + 4\eta^2 r \frac{M^+}{N} \Sigma_L. \tag{50}$$

Since $\eta S r \le 1/4$, we have $16\eta^4 r^4 S^2 = (4\eta S r)^2 \, \eta^2 r^2 \le \eta^2 r^2$ and hence $16\eta^4 r^4 S^2 \Theta \le \eta^2 r^2 \Theta$. Combining (37) and (50) with $\mathbb{E}[D^t] \le S^2 \, \mathbb{E}[\Phi_W^t + \Phi_C^t]$, we obtain

$$\mathbb{E}[D^t] \le 2S^2 \eta^2 r^2 \Theta + 16 S^2 \eta^2 r^2 \zeta^2 + 4 S^2 \eta^2 r \frac{M^+}{N} \Sigma_L. \tag{51}$$

Substituting (51) into (29), summing over $t = 1, \ldots, T$, using $F(\bar{\boldsymbol{\omega}}^T) \ge F^\star$, and dividing by $\eta T/4$, we obtain (23). This completes the proof. ∎

Theorem 1 establishes a stationarity bound for the virtual average sequence. The last three terms of (23) grow with the aggregation interval E and capture the cost of less frequent fed server aggregation. Among them, the splitting and offloading drift and the between-edge noise further depend on the decisions, through $\Theta(\boldsymbol{\mu}, \boldsymbol{\beta})$ and M^+. Specifically, between two consecutive fed-server aggregations, each edge server m only averages the layers above the deepest split layer $L_m^\star$ of its group. The first $L_m^\star$ layers of every device in group m are therefore updated for up to $E-1$ rounds without synchronization, and $\Theta(\boldsymbol{\mu}, \boldsymbol{\beta})$ in (21) accumulates the corresponding layer-wise gradient bounds. In particular, assigning a device with a deeper split layer to edge server m increases $L_m^\star$ and thus the drift of every device in the group. This coupling is the reason why layer splitting and offloading should be optimized jointly.

B. Latency Analysis

In this subsection, we analyze the latency during split training and periodic federated aggregation, including both computation and communication parts. Within each group, the

edge server waits for the assigned devices to upload their split-layer activations, continues the FP for each device, and aggregates the common part of the server-side sub-models. After E rounds of split training, all devices and edge servers upload the relevant sub-models to the fed server for aggregation.

1) Device-side FP Latency: Let ρ_l denote the computational workload of FP through the first l layers per data sample, measured in CPU cycles. The device-side FP latency is

$$T_n^{\mathrm{FP}} = \frac{B}{f_n} \sum_{l \in \mathcal{L}} \mu_{n,l} \rho_l, \ n \in \mathcal{N}, \tag{52}$$

where f_n is the computation capability of device n in CPU cycles per second.

2) Activation Transmission: We consider activation transmission from devices to edge servers over orthogonal channels. Let $h_{n,m}$ denote the channel gain from device n to edge server m, σ_0^2 denote the received noise power, P denote the transmit power of a device, and W denote the bandwidth allocated to a channel. The uplink rate from device n to edge server m is given by $r_{n,m}^{\mathrm{UL}} = W \log_2(1 + P|h_{n,m}|^2/\sigma_0^2)$, $n \in \mathcal{N}$, $m \in \mathcal{M}$. Let ψ_l denote the activation size at split layer l. The activation transmission latency is

$$T_{n,m}^{\mathrm{UL}} = \frac{B}{r_{n,m}^{\mathrm{UL}}} \sum_{l \in \mathcal{L}} \mu_{n,l} \psi_l, \ n \in \mathcal{N}, \ m \in \mathcal{M}. \tag{53}$$

3) Server-side FP and BP Latency: The server-side FP latency at edge server m is

$$T_m^{\mathrm{FP}} = \frac{B}{f_m} \sum_{n=1}^{N} \beta_{n,m} \sum_{l \in \mathcal{L}} \mu_{n,l} (\rho_L - \rho_l), \ m \in \mathcal{M}, \tag{54}$$

where f_m is the computation capability of edge server m. Let α_l denote the BP workload through the first l layers per data sample. The server-side BP latency is

$$T_m^{\mathrm{BP}} = \frac{B}{f_m} \sum_{n=1}^{N} \beta_{n,m} \sum_{l \in \mathcal{L}} \mu_{n,l} (\alpha_L - \alpha_l), \ m \in \mathcal{M}. \tag{55}$$

4) Intermediate Gradient Transmission: Let λ_l denote the size of the intermediate gradient at split layer l. The downlink transmission latency from edge server m to device n is

$$T_{m,n}^{\mathrm{DL}} = \frac{B}{r_{m,n}^{\mathrm{DL}}} \sum_{l \in \mathcal{L}} \mu_{n,l} \lambda_l, \ n \in \mathcal{N}, \ m \in \mathcal{M}, \tag{56}$$

where $r_{m,n}^{\mathrm{DL}} = W \log_2(1 + P_{\mathrm{s}}|h_{n,m}|^2/\sigma_0^2)$ is the downlink rate from edge server m to device n and P_{s} is the transmit power of an edge server.

5) Device-side BP Latency: The device-side BP latency is

$$T_n^{\mathrm{BP}} = \frac{B}{f_n} \sum_{l \in \mathcal{L}} \mu_{n,l} \alpha_l, \ n \in \mathcal{N}. \tag{57}$$

6) Latency during Federated Aggregation: Let δ_l denote the data size of the device-side sub-model for split layer l. Let $r_{n,\mathrm{fed}}^{\mathrm{UL}}$, $r_{m,\mathrm{fed}}^{\mathrm{UL}}$, $r_{\mathrm{fed},n}^{\mathrm{DL}}$, and $r_{\mathrm{fed},m}^{\mathrm{DL}}$ denote the uplink and downlink rates between device n, edge server m, and the fed server. The device-side and server-side uplink latencies are

$$T_{n,\mathrm{fed}}^{\mathrm{UL}} = \frac{\sum_{l \in \mathcal{L}} \mu_{n,l} \delta_l}{r_{n,\mathrm{fed}}^{\mathrm{UL}}}, \ n \in \mathcal{N} \tag{58}$$

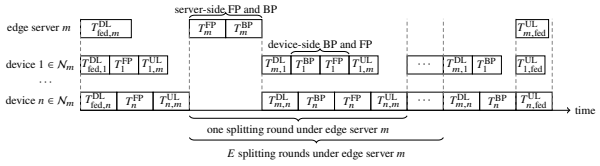


Fig. 2. Illustration of the training latency T_m in (62) within each group m.

and

$$T_{m,\mathrm{fed}}^{\mathrm{UL}} = \frac{\sum_{n=1}^{N} \beta_{n,m} \sum_{l \in \mathcal{L}} \mu_{n,l} (\delta_L - \delta_l)}{r_{m,\mathrm{fed}}^{\mathrm{UL}}}, \ m \in \mathcal{M}. \tag{59}$$

The corresponding downlink latencies are

$$T_{\mathrm{fed},n}^{\mathrm{DL}} = \frac{\sum_{l \in \mathcal{L}} \mu_{n,l} \delta_l}{r_{\mathrm{fed},n}^{\mathrm{DL}}}, \ n \in \mathcal{N} \tag{60}$$

and

$$T_{\mathrm{fed},m}^{\mathrm{DL}} = \frac{\sum_{n=1}^{N} \beta_{n,m} \sum_{l \in \mathcal{L}} \mu_{n,l} (\delta_L - \delta_l)}{r_{\mathrm{fed},m}^{\mathrm{DL}}}, \ m \in \mathcal{M}. \tag{61}$$

The aggregation computation delay at the fed server is much smaller than the transmission and local training delays, and hence is not included.

7) Total Training Latency: For edge server m, the latency over one synchronization period, illustrated in Fig. 2, is

$$\begin{aligned} T_m = &\max\{T_{\mathrm{fed},m}^{\mathrm{DL}}, \ \max_{n \in \mathcal{N}} \beta_{n,m} (T_{\mathrm{fed},n}^{\mathrm{DL}} + T_n^{\mathrm{FP}} + T_{n,m}^{\mathrm{UL}})\} \\ &+ (E-1)[T_m^{\mathrm{FP}} + T_m^{\mathrm{BP}} \\ &+ \max_{n \in \mathcal{N}} \beta_{n,m} (T_{m,n}^{\mathrm{DL}} + T_n^{\mathrm{BP}} + T_n^{\mathrm{FP}} + T_{n,m}^{\mathrm{UL}})] \\ &+ T_m^{\mathrm{FP}} + T_m^{\mathrm{BP}} \\ &+ \max\{T_{m,\mathrm{fed}}^{\mathrm{UL}}, \ \max_{n \in \mathcal{N}} \beta_{n,m} (T_{m,n}^{\mathrm{DL}} + T_n^{\mathrm{BP}} + T_{n,\mathrm{fed}}^{\mathrm{UL}})\}. \end{aligned} \tag{62}$$

This expression is well defined when an edge server is inactive, with the convention that empty sums are zero. The total latency for T training rounds is

$$T^{\mathrm{total}}(E, \boldsymbol{\mu}, \boldsymbol{\beta}) = \frac{T}{E} \max_{m \in \mathcal{M}} T_m. \tag{63}$$

For the optimization, we use (63) as the synchronization-period model. If T is not divisible by E, replacing T/E by $\lceil T/E \rceil$ accounts for the final partial period as a complete synchronization period and therefore gives a conservative latency upper bound.

For the label-on-device variant in Section II, edge server m sends the penultimate activations to device n and receives the gradients of the same size in each round. This adds $\Delta T_{n,m} = B\psi_{L-1}(1/r_{m,n}^{\mathrm{DL}} + 1/r_{n,m}^{\mathrm{UL}})$ to the latency of device n in each round of (62). The output layer is then no longer averaged at the edge server in each round, which adds the decision-independent constant G_L^2 to $\Theta(\boldsymbol{\mu}, \boldsymbol{\beta})$ in (21). For ResNet-18, the relative payload overhead $2\psi_{L-1}/(\psi_l + \lambda_l)$ is 3.1% at $l = 1$, 12.5% at $l \in \{2, 3\}$, 25% at $l = 4$, 50% at $l = 5$, and 100% at $l \in \{6, 7, 8\}$. Under the rates in Section V-A, $\Delta T_{n,m} \leq 10.2$ ms per round.

C. Problem Formulation

Given the convergence and latency analyses, we first formulate the following problem to minimize the average stationarity measure of the training process subject to the latency and memory constraints:

$$\mathbf{P}_1: \quad \underset{E,\boldsymbol{\mu},\boldsymbol{\beta}}{\text{minimize}} \frac{1}{T}\sum_{t=1}^{T}\mathbb{E}\|\nabla F(\bar{\boldsymbol{\omega}}^{t-1})\|^2 \tag{64a}$$

$$\text{subject to } T^{\text{total}}(E,\boldsymbol{\mu},\boldsymbol{\beta}) \le T^{\max}, \tag{64b}$$

$$\sum_{l\in\mathcal{L}}\mu_{n,l}\theta_l \le C_n,\ n\in\mathcal{N}, \tag{64c}$$

$$\sum_{n=1}^{N}\beta_{n,m}\sum_{l\in\mathcal{L}}\mu_{n,l}(\theta_L-\theta_l) \le C_m, m\in\mathcal{M}, \tag{64d}$$

$$\sum_{l\in\mathcal{L}}\mu_{n,l}=1,\ n\in\mathcal{N}, \tag{64e}$$

$$\sum_{m=1}^{M}\beta_{n,m}=1,\ n\in\mathcal{N}, \tag{64f}$$

$$\mu_{n,l}\in\{0,1\},\ n\in\mathcal{N},\ l\in\mathcal{L}, \tag{64g}$$

$$\beta_{n,m}\in\{0,1\},\ n\in\mathcal{N},\ m\in\mathcal{M}, \tag{64h}$$

$$E\in\mathbb{N}^+, \tag{64i}$$

$$E\le\min\{E^{\text{th}},T\}. \tag{64j}$$

The learning rate η is fixed and satisfies $\eta \le 1/(4S)$. The threshold

$$E^{\text{th}} = \left\lfloor 1+\frac{1}{4\eta S}\right\rfloor \tag{65}$$

enforces the second step-size condition in Theorem 1, and it serves as the upper limit of the search range of the aggregation interval in the optimization. Constraints (64c) and (64d) represent the memory capacities of devices and edge servers. The parameter θ_l is an upper estimate of the memory required to train the first l layers of the model. It covers the storage of the sub-model parameters, the activations, and the gradients.

Theorem 1 provides an upper bound for the objective of $\mathbf{P}_1$. Removing those terms which are independent of $(E,\boldsymbol{\mu},\boldsymbol{\beta})$ and dividing by the positive constant $6S^2\eta^2$, we obtain the following bound-based surrogate:

$$\begin{aligned} J(E,\boldsymbol{\mu},\boldsymbol{\beta},\boldsymbol{\chi}) = (E-1)^2\left[\Theta(\boldsymbol{\mu},\boldsymbol{\beta})+8\zeta^2\right] \\ +2(E-1)\frac{\Sigma_L}{N}\sum_{m=1}^{M}\chi_m. \end{aligned} \tag{66}$$

Here, $\chi_m\in\{0,1\}$ indicates whether edge server m is active. Hence,

$$M^+ = |\mathcal{M}^+| = \sum_{m=1}^{M}\chi_m. \tag{67}$$

The objective in (66) keeps the last term of (23), which depends on the number of active edge servers. When the latency and memory constraints permit, this term favors offloading decisions that activate fewer edge servers, whereas $\Theta(\boldsymbol{\mu},\boldsymbol{\beta})$ favors more edge servers, since smaller groups have a smaller $L_m^\star$. Minimizing J balances these two effects against the latency budget. This behavior is consistent with the between-edge noise term in Theorem 1. A conservative alternative is to replace $\sum_{m=1}^{M}\chi_m$ with the constant M, since $M^+\le M$. This alternative still yields a valid upper bound, but the resulting surrogate is looser and no longer reflects the offloading decisions in this term.

The function $\Theta(\boldsymbol{\mu},\boldsymbol{\beta})$ in (21) contains the maximum split layer within each edge group. To encode it, we introduce $y_{m,l}\in\{0,1\}$, which indicates whether $L_m^\star = l$. Together with χ_m, we impose

$$\sum_{l\in\mathcal{L}} y_{m,l} = \chi_m, \qquad m\in\mathcal{M}, \tag{68}$$

$$\chi_m \le \sum_{n=1}^{N}\beta_{n,m} \le N\chi_m, \qquad m\in\mathcal{M}. \tag{69}$$

Introduce $\gamma_{n,m,l}\in[0,1]$ to represent $\beta_{n,m}\mu_{n,l}$:

$$0\le\gamma_{n,m,l}\le 1, \tag{70a}$$

$$\gamma_{n,m,l}\le\beta_{n,m}, \tag{70b}$$

$$\gamma_{n,m,l}\le\mu_{n,l}, \tag{70c}$$

$$\gamma_{n,m,l}\ge\beta_{n,m}+\mu_{n,l}-1, \tag{70d}$$

for all $n\in\mathcal{N}$, $m\in\mathcal{M}$, and $l\in\mathcal{L}$. The maximum split layer is enforced by

$$\sum_{k=l}^{L-1} y_{m,k} \ge \gamma_{n,m,l}, \qquad n\in\mathcal{N},\ m\in\mathcal{M},\ l\in\mathcal{L}, \tag{71}$$

and

$$y_{m,l} \le \sum_{n=1}^{N}\gamma_{n,m,l}, \qquad m\in\mathcal{M},\ l\in\mathcal{L}. \tag{72}$$

Constraints (68), (69), (71), and (72) imply that $y_{m,l}=1$ when l is the maximum split layer among devices assigned to edge server m. Introduce $\upsilon_{n,m,l}\in[0,1]$ to represent $\beta_{n,m}y_{m,l}$:

$$0\le\upsilon_{n,m,l}\le 1, \quad n\in\mathcal{N},\ m\in\mathcal{M},\ l\in\mathcal{L}, \tag{73a}$$

$$\upsilon_{n,m,l}\le\beta_{n,m}, \quad n\in\mathcal{N},\ m\in\mathcal{M},\ l\in\mathcal{L}, \tag{73b}$$

$$\upsilon_{n,m,l}\le y_{m,l}, \quad n\in\mathcal{N},\ m\in\mathcal{M},\ l\in\mathcal{L}, \tag{73c}$$

$$\upsilon_{n,m,l}\ge\beta_{n,m}+y_{m,l}-1, \quad n\in\mathcal{N},\ m\in\mathcal{M},\ l\in\mathcal{L}. \tag{73d}$$

Then

$$\Theta(\boldsymbol{\mu},\boldsymbol{\beta}) = \frac{1}{N}\sum_{m=1}^{M}\sum_{n=1}^{N}\sum_{l\in\mathcal{L}} H_l\upsilon_{n,m,l}. \tag{74}$$

The optimization problem that minimizes the bound-based surrogate J is

$$\mathbf{P}_2: \quad \underset{E,\boldsymbol{\mu},\boldsymbol{\beta},\boldsymbol{\chi},\boldsymbol{y},\boldsymbol{\gamma},\boldsymbol{\upsilon}}{\text{minimize}} J(E,\boldsymbol{\mu},\boldsymbol{\beta},\boldsymbol{\chi}) \tag{75a}$$

$$\text{subject to} \quad \text{constraints (64b)}-\text{(64j)}, \text{(68)}-\text{(73)},$$

$$\chi_m\in\{0,1\}, \qquad m\in\mathcal{M}, \tag{75b}$$

$$y_{m,l}\in\{0,1\}, \qquad m\in\mathcal{M},\ l\in\mathcal{L}. \tag{75c}$$

Problem $\mathbf{P}_2$ minimizes a valid convergence-bound surrogate rather than an equivalent expression for $\mathbf{P}_1$. This distinction is important because Theorem 1 gives an upper bound on the stationarity measure.

The latency constraint in (64b) contains the maximum operator. We introduce continuous variables a_m, b_m, and c_m to

represent the three maximum terms in (62). Then the latency constraint can be rewritten as

$$
\begin{aligned}
&a_m + (E-1)b_m + c_m \\
&\quad + \frac{EB}{f_m}\sum_{n=1}^{N}\sum_{l\in\mathcal{L}}\gamma_{n,m,l}(\rho_L - \rho_l + \alpha_L - \alpha_l) \\
&\quad \le \frac{T^{\max}E}{T}, \quad m \in \mathcal{M}, && \text{(76a)}\\
&a_m \ge \frac{\sum_{n=1}^{N}\sum_{l\in\mathcal{L}}\gamma_{n,m,l}(\delta_L - \delta_l)}{r^{\mathrm{DL}}_{\mathrm{fed},m}}, \quad m \in \mathcal{M}, && \text{(76b)}\\
&a_m \ge \sum_{l\in\mathcal{L}}\gamma_{n,m,l}\left(\frac{\delta_l}{r^{\mathrm{DL}}_{\mathrm{fed},n}} + \frac{B\rho_l}{f_n} + \frac{B\psi_l}{r^{\mathrm{UL}}_{n,m}}\right), \\
&\qquad\qquad n \in \mathcal{N},\ m \in \mathcal{M}, && \text{(76c)}\\
&b_m \ge \sum_{l\in\mathcal{L}}\gamma_{n,m,l}\left(\frac{B\lambda_l}{r^{\mathrm{DL}}_{m,n}} + \frac{B(\alpha_l+\rho_l)}{f_n} + \frac{B\psi_l}{r^{\mathrm{UL}}_{n,m}}\right), \\
&\qquad\qquad n \in \mathcal{N},\ m \in \mathcal{M}, && \text{(76d)}\\
&c_m \ge \frac{\sum_{n=1}^{N}\sum_{l\in\mathcal{L}}\gamma_{n,m,l}(\delta_L - \delta_l)}{r^{\mathrm{UL}}_{m,\mathrm{fed}}}, \quad m \in \mathcal{M}, && \text{(76e)}\\
&c_m \ge \sum_{l\in\mathcal{L}}\gamma_{n,m,l}\left(\frac{B\lambda_l}{r^{\mathrm{DL}}_{m,n}} + \frac{B\alpha_l}{f_n} + \frac{\delta_l}{r^{\mathrm{UL}}_{n,\mathrm{fed}}}\right), \\
&\qquad\qquad n \in \mathcal{N},\ m \in \mathcal{M}, && \text{(76f)}
\end{aligned}
$$

Similarly, the edge-server memory constraint becomes

$$
\sum_{n=1}^{N}\sum_{l\in\mathcal{L}}\gamma_{n,m,l}(\theta_L - \theta_l) \le C_m, \qquad m \in \mathcal{M}. \tag{77}
$$

With the auxiliary variables $\boldsymbol{a}$, $\boldsymbol{b}$, $\boldsymbol{c}$, $\boldsymbol{\gamma}$, and $\boldsymbol{\upsilon}$, problem $\mathbf{P}_2$ can equivalently be reformulated as

$$
\begin{aligned}
\mathbf{P}_3: \quad &\underset{E,\boldsymbol{\mu},\boldsymbol{\beta},\boldsymbol{\chi},\boldsymbol{y},\boldsymbol{\gamma},\boldsymbol{\upsilon},\boldsymbol{a},\boldsymbol{b},\boldsymbol{c}}{\text{minimize}} && J(E,\boldsymbol{\mu},\boldsymbol{\beta},\boldsymbol{\chi}) && \text{(78a)}\\
&\text{subject to} && \text{constraints (64c), (64e),} \\
& && \text{(64f)}-\text{(64j), (68)}-\text{(73),} \\
& && \text{(75b), (75c), (76), (77),} \\
& && a_m, b_m, c_m \ge 0, \qquad m \in \mathcal{M}. && \text{(78b)}
\end{aligned}
$$

Problem $\mathbf{P}_3$ is a mixed-integer nonlinear program because the objective contains products between $(E-1)^2$ and assignment-dependent variables, and the latency constraint contains products between E and assignment-dependent variables. We next develop an efficient solution approach.

IV. Solution Approach

In this section, we solve the MINLP problem $\mathbf{P}_3$. We first decompose problem $\mathbf{P}_3$ over the finite set of admissible aggregation intervals and formulate a fixed-interval MILP for each candidate value. We then derive an interval-level lower bound and develop an exact algorithm that prunes unnecessary subproblems while preserving global optimality. Finally, we introduce feasible warm starts and group-configuration cuts to accelerate the solution of the fixed-interval MILPs.

A. Exact Finite-Interval Decomposition

After the auxiliary-variable reformulation in Section III-C, all remaining nonlinear terms in $\mathbf{P}_3$ involve the scalar aggregation interval E. Constraints (64i) and (64j) confine the aggregation interval E to the finite set

$$
\bar{E} \triangleq \min\{E^{\mathrm{th}}, T\}, \quad \mathcal{E} \triangleq \{1, 2, \ldots, \bar{E}\}. \tag{79}
$$

For a fixed aggregation interval $E = e \in \mathcal{E}$, we substitute Θ in its linearized form (74) into the surrogate (66) and collect the decision-dependent terms into

$$
\begin{aligned}
\widetilde{J}_e(\boldsymbol{\upsilon},\boldsymbol{\chi}) = (e-1)^2\frac{1}{N}\sum_{m=1}^{M}\sum_{n=1}^{N}\sum_{l\in\mathcal{L}}H_l\upsilon_{n,m,l} \\
+ 2(e-1)\frac{\Sigma_L}{N}\sum_{m=1}^{M}\chi_m.
\end{aligned} \tag{80}
$$

The surrogate value for interval e is then

$$
J_e(\boldsymbol{\upsilon},\boldsymbol{\chi}) = \widetilde{J}_e(\boldsymbol{\upsilon},\boldsymbol{\chi}) + 8(e-1)^2\zeta^2. \tag{81}
$$

The last term $8(e-1)^2\zeta^2$ does not depend on the decisions once e is fixed. It is therefore omitted when solving the corresponding subproblem and restored when different intervals are compared.

For each $e \in \mathcal{E}$, consider the following fixed-interval problem:

$$
\begin{aligned}
\mathbf{P}_4(e): \quad &\underset{\boldsymbol{\mu},\boldsymbol{\beta},\boldsymbol{\chi},\boldsymbol{y},\boldsymbol{\gamma},\boldsymbol{\upsilon},\boldsymbol{a},\boldsymbol{b},\boldsymbol{c}}{\text{minimize}} && \widetilde{J}_e(\boldsymbol{\upsilon},\boldsymbol{\chi}) && \text{(82a)}\\
&\text{subject to} && \text{(64c), (64e), (64f)–(64h),} \\
& && \text{(68)–(73), (75b), (75c), (78b),} \\
& && \text{(76), (77),} \\
& && \text{with } E \text{ fixed to } e \text{ in (76a).} && \text{(82b)}
\end{aligned}
$$

For fixed e, all coefficients in the objective and constraints are constants. Hence, $\mathbf{P}_4(e)$ is a MILP and can be solved by a standard branch-and-cut solver such as Gurobi [24].

Let $V_e^\star$ denote the optimal objective value of $\mathbf{P}_4(e)$, with $V_e^\star = +\infty$ when $\mathbf{P}_4(e)$ is infeasible. Define the corresponding full surrogate value as

$$
Z_e^\star \triangleq V_e^\star + 8(e-1)^2\zeta^2. \tag{83}
$$

Proposition 1 (Optimality of the finite-interval decomposition)**.** *The optimal value of* $\mathbf{P}_3$ *is*

$$
Z^\star = \min_{e\in\mathcal{E}} Z_e^\star. \tag{84}
$$

If $e^\star \in \arg\min_{e\in\mathcal{E}} Z_e^\star$*, then any optimizer of* $\mathbf{P}_4(e^\star)$*, together with* $E = e^\star$*, is globally optimal for* $\mathbf{P}_3$*. If* $Z_e^\star = +\infty$ *for all* $e \in \mathcal{E}$*, then* $\mathbf{P}_3$ *is infeasible.*

Proof: Let $\boldsymbol{z} \triangleq (\boldsymbol{\mu},\boldsymbol{\beta},\boldsymbol{\chi},\boldsymbol{y},\boldsymbol{\gamma},\boldsymbol{\upsilon},\boldsymbol{a},\boldsymbol{b},\boldsymbol{c})$ collect all decision variables of $\mathbf{P}_3$ other than E. Let $\mathcal{F}$ denote the feasible set of $\mathbf{P}_3$, and let $\mathcal{F}_e$ denote the feasible set obtained by fixing $E = e$. Since E is an integer and satisfies $E \in \mathcal{E}$, we have $\mathcal{F} = \bigcup_{e\in\mathcal{E}}(\{e\} \times \mathcal{F}_e)$. For fixed e, substituting $E = e$ into $\mathbf{P}_3$ removes all products involving E and yields exactly the

constraints of $\mathbf{P}_4(e)$. Moreover, for every point in $\mathcal{F}_e$, the objective of $\mathbf{P}_3$ equals $\widetilde{J}_e + 8(e-1)^2\zeta^2$. Therefore,

$$\begin{aligned}\min_{(E,\boldsymbol{z})\in\mathcal{F}} J &= \min_{e\in\mathcal{E}} \left\{ \min_{\boldsymbol{z}\in\mathcal{F}_e} \widetilde{J}_e + 8(e-1)^2\zeta^2 \right\} \\ &= \min_{e\in\mathcal{E}} Z_e^\star.\end{aligned} \tag{85}$$

The optimizer correspondence and the infeasibility statement follow directly. ∎

When $e = 1$, both terms in (80) and the constant in (81) are zero. Hence, $\mathbf{P}_4(1)$ is a feasibility problem, and any feasible solution at $e = 1$ is globally optimal for the surrogate. In this case, the latency budget permits fed server aggregation in every round. Because the surrogate does not rank the feasible layer splitting and offloading decisions, they can be selected by a secondary criterion such as the total latency in (63).

B. Interval Pruning and Exact/Anytime Algorithm

The finite-interval decomposition is exact, but solving every fixed-interval MILP may be unnecessary. The following interval-level lower bound allows the search to terminate early.

Lemma 1 (Interval pruning bound)**.** *For every $e \in \mathcal{E}$ and every feasible point of $\mathbf{P}_4(e)$,*

$$J_e \geq \mathrm{LB}(e) \triangleq (e-1)^2\left(H_1 + 8\zeta^2\right) + 2(e-1)\frac{\Sigma_L}{N}. \tag{86}$$

The function $\mathrm{LB}(e)$ is nondecreasing in e. It is strictly increasing if $H_1 + 8\zeta^2 > 0$ or $\Sigma_L > 0$.

Proof: Constraint (64f) guarantees that each device is assigned to exactly one edge server. Since $N \geq 1$, at least one edge server is active, and the active-edge encoding implies $\sum_{m=1}^{M} \chi_m \geq 1$. For each active edge server m, its maximum selected split layer satisfies $L_m^\star \geq 1$. Since $H_l = \sum_{k=1}^{l} G_k^2$ is nondecreasing in l, we have $\Theta(\boldsymbol{\mu}, \boldsymbol{\beta}) = \sum_{m\in\mathcal{M}^+} \frac{N_m}{N} H_{L_m^\star} \geq H_1 \sum_{m\in\mathcal{M}^+} \frac{N_m}{N} = H_1$. Substituting these two inequalities into (81) gives (86). Finally, $\mathrm{LB}(e+1) - \mathrm{LB}(e) = (2e-1)(H_1 + 8\zeta^2) + 2\Sigma_L/N \geq 0$, which proves the monotonicity and the stated strictness condition. ∎

Algorithm 2 processes the intervals in increasing order and maintains the best feasible full surrogate value $\overline{Z}$. If $\mathrm{LB}(e) \geq \overline{Z}$, then Lemma 1 and the monotonicity of $\mathrm{LB}(e)$ imply that interval e and all larger intervals cannot improve the incumbent.

If every visited subproblem is solved to global optimality or proved infeasible, and every unvisited interval is pruned by Lemma 1, then $\underline{Z} = \overline{Z} = Z^\star$, and Proposition 1 gives a globally optimal solution of $\mathbf{P}_3$. Under a solver time limit, an interval is marked infeasible only when infeasibility is proven. Otherwise, the lower bound obtained by the solver is retained in Algorithm 2. Since every interval has a valid lower bound and $\overline{Z}$ is the value of a feasible solution,

$$\underline{Z} \leq Z^\star \leq \overline{Z}. \tag{87}$$

Thus, $\overline{Z} - \underline{Z}$ is a valid absolute optimality gap for the instantiated surrogate. If no feasible incumbent is found under the latency budget, the algorithm does not declare infeasibility unless all intervals have been proven to be infeasible.

Algorithm 2 Exact/Anytime Finite-Interval MILP Algorithm for MESFL Optimization

Input: Problem parameters and $\mathcal{E} = \{1, \ldots, \bar{E}\}$
Output: Best feasible solution $\overline{\mathcal{X}}$ and bounds $\underline{Z}, \overline{Z}$
Set $\overline{Z} := +\infty$ and $\overline{\mathcal{X}} := \emptyset$
Set $\underline{Z}_e := \mathrm{LB}(e)$ for all $e \in \mathcal{E}$
for each $e \in \mathcal{E}$ in increasing order **do**
  if $\mathrm{LB}(e) \geq \overline{Z}$ **then**
    break
  end if
  Construct $\mathbf{P}_4(e)$ and optionally provide a verified feasible warm start
  Solve $\mathbf{P}_4(e)$, obtaining a valid lower bound $\underline{V}_e$ and, if available, a feasible incumbent value $\overline{V}_e$
  if $\mathbf{P}_4(e)$ is proved to be infeasible **then**
    Set $\underline{Z}_e := +\infty$
  else
    Set $\underline{Z}_e := \max\{\mathrm{LB}(e), \underline{V}_e + 8(e-1)^2\zeta^2\}$
    if a feasible incumbent is available **then**
      Set $\overline{Z}_e := \overline{V}_e + 8(e-1)^2\zeta^2$
      if $\overline{Z}_e < \overline{Z}$ **then**
        Set $\overline{Z} := \overline{Z}_e$ and store the corresponding solution in $\overline{\mathcal{X}}$
      end if
    end if
  end if
end for
Set $\underline{Z} := \min_{e\in\mathcal{E}} \underline{Z}_e$
Return: $\overline{\mathcal{X}}$, $\underline{Z}$, and $\overline{Z}$

With $|\mathcal{L}| = L - 1$ candidate split layers, each fixed-interval problem $\mathbf{P}_4(e)$ contains $(N+M)|\mathcal{L}| + NM + M$ binary variables, $2NM|\mathcal{L}| + 3M$ continuous variables, and $7NM|\mathcal{L}| + 3NM + M|\mathcal{L}| + 7M + 3N$ linear constraints, where the box constraints on $\boldsymbol{\gamma}$ and $\boldsymbol{\upsilon}$ and the nonnegativity constraints on $\boldsymbol{a}$, $\boldsymbol{b}$, and $\boldsymbol{c}$ are treated as variable bounds. The dominant $7NM|\mathcal{L}|$ part comes from the product linearizations (70) and (73) and the maximum-layer constraint (71). Algorithm 2 solves at most $\bar{E}$ MILPs. Their worst-case branch-and-cut complexity is exponential [25], while valid solver bounds permit exact or anytime operation. The fixed-interval subproblems are mutually independent and can be solved in parallel. Our sequential increasing-order implementation instead exploits Lemma 1. A batched parallel implementation trades some pruning efficiency for lower wall-clock time.

C. Warm Starts and Group-Configuration Cuts

The following mechanisms accelerate the fixed-interval MILPs without changing their integer feasible sets or the optimality result in Proposition 1.

For warm starting, we solve the linear programming (LP) relaxation of $\mathbf{P}_4(e)$ and round the split-layer variables $\boldsymbol{\mu}$ and the offloading variables $\boldsymbol{\beta}$ to integer values. The rounded point may violate the memory constraints (64c) and (77) or the latency constraints (76). In that case, we repair it by solving a feasibility MILP over the constraints of $\mathbf{P}_4(e)$, which returns

an integer point that satisfies all these constraints. A feasible incumbent from a neighboring interval may also be reused, but only after its feasibility with respect to (76) and (77) is verified for the new value of e. Otherwise, it is repaired in the same way or discarded.

A second acceleration mechanism exploits the fact that the group latency in (62) cannot decrease when more device–split pairs are assigned to the group. This monotonicity is established in the proof of Lemma 2. For a fixed interval e, consider an integer candidate $(\bar{\boldsymbol{\mu}}, \bar{\boldsymbol{\beta}})$ and define

$$\bar{l}_n \triangleq \sum_{l\in\mathcal{L}} l\bar{\mu}_{n,l}, \quad \bar{\mathcal{N}}_m \triangleq \{n \in \mathcal{N} \mid \bar{\beta}_{n,m} = 1\}. \tag{88}$$

If group m violates the latency budget when evaluated using (62), we define its device–split configuration as

$$\mathcal{C}_m \triangleq \{(n, \bar{l}_n) \mid n \in \bar{\mathcal{N}}_m\}. \tag{89}$$

We then add the group-configuration cut

$$\sum_{(n,l)\in\mathcal{C}_m} (1 - \gamma_{n,m,l}) \geq 1. \tag{90}$$

The cut requires at least one device in the violating group to change either its edge server assignment or its split layer.

Lemma 2 (Validity of the group-configuration cut)**.** *For the interval e that generates $\mathcal{C}_m$, the inequality* (90) *excludes no feasible integer point of problem $\mathbf{P}_4(e)$. Moreover, it excludes every integer assignment whose group-m device–split configuration contains $\mathcal{C}_m$.*

Proof: Fix e and consider two group-m configurations $\mathcal{C}_m \subseteq \mathcal{C}'_m$. In (62), the server-side computation and fed-server exchange terms are sums of nonnegative contributions over the assigned device–split pairs. The remaining waiting-time terms are maxima over nonnegative pair-specific quantities. Hence, enlarging the configuration cannot decrease the group latency, i.e., $T_m(e; \mathcal{C}'_m) \geq T_m(e; \mathcal{C}_m)$. Because $\mathcal{C}_m$ violates the latency budget, every configuration containing $\mathcal{C}_m$ is also infeasible for problem $\mathbf{P}_4(e)$.

At every integer feasible point, $\boldsymbol{\mu}$ and $\boldsymbol{\beta}$ are binary vectors, and the exact product-linearization constraints enforce $\gamma_{n,m,l} = \beta_{n,m}\mu_{n,l}$. Therefore, an assignment contains all pairs in $\mathcal{C}_m$ if and only if $\gamma_{n,m,l} = 1$ for every $(n, l) \in \mathcal{C}_m$, in which case the left-hand side of (90) is zero and the assignment is excluded. Conversely, every feasible assignment must omit at least one pair in $\mathcal{C}_m$ and therefore satisfies (90). This proves both claims. ∎

The group-configuration cut in (90) is interval-specific because the latency violation depends on e. It can be generated from an infeasible integer warm-start candidate or in a callback that checks integer assignments using (62). Unlike a cut that excludes only one complete assignment, it removes all assignments that retain the same overloaded group configuration.

D. Calibration of the Bound Constants

Theorem 1 depends on the constants $\{G_k^2\}$ (through $H_l = \sum_{k=1}^{l} G_k^2$), Σ_L, ζ^2, and S, which are properties of the architecture, the data, and the mini-batch size and are not known *a priori*. We estimate them by a one-time warm-up before training: each device n evaluates $W = 8$ probe mini-batches at the initial model $\boldsymbol{\omega}^0$, which gives a set $\mathcal{W}_n$ of $|\mathcal{W}_n| = W$ (parameter, mini-batch) pairs, and reports its per-layer average squared gradient norms over the probes. The decision module then takes the maximum of these reports over all devices and inflates it by a safety margin κ, namely

$$\widehat{G}_k^2 = (1+\kappa) \max_{n\in\mathcal{N}} \frac{1}{|\mathcal{W}_n|} \sum_{(\boldsymbol{\omega},\xi_n)\in\mathcal{W}_n} \left\|\mathbf{g}_{n,k}(\boldsymbol{\omega}, \xi_n)\right\|^2, \tag{91}$$

where the margin $\kappa = 0.2$ reduces the risk of underestimating the true constant G_k^2 in Assumption 3, and $\widehat{H}_l = \sum_{k=1}^{l} \widehat{G}_k^2$. The constants $\widehat{\Sigma}_L$ and $\widehat{\zeta}^2$ are estimated in the same way, with the same margin, from the empirical mini-batch gradient variance, obtained by summing the per-layer variances reported by each device, and from the cross-device dispersion $\frac{1}{N}\sum_n \|\mathbf{g}_n - \bar{\mathbf{g}}\|^2$ over the probes. The smoothness constant is estimated by a finite difference of mini-batch gradients along the training trajectory,

$$\widehat{S} = (1+\kappa) \max_{n\in\mathcal{N}} \max_{(\boldsymbol{\omega},\xi_n)\in\mathcal{W}_n} \frac{\|\mathbf{g}_n(\boldsymbol{\omega} - \eta\mathbf{g}_n, \xi_n) - \mathbf{g}_n\|}{\eta\|\mathbf{g}_n\|}, \tag{92}$$

where $\mathbf{g}_n$ is short for $\mathbf{g}_n(\boldsymbol{\omega}, \xi_n)$, the secant is taken along the local gradient direction with the step size η, and both gradients use the same mini-batch ξ_n, which removes the sampling noise so that the quotient reflects the curvature of the local loss along the trajectory. The surrogate (66) does not depend on S, since S cancels in the division by $6S^2\eta^2$. The estimate $\widehat{S}$ therefore acts only through the threshold E^{th} in (65), where over-estimating S shrinks the admissible interval set $\mathcal{E}$ in (79) and is again conservative.[4]

The estimates in (91) and (92) are finite-sample values at the initial model and are not guaranteed to satisfy Assumptions 1–4 uniformly. Therefore, problem $\mathbf{P}_3$ with these estimates is a calibrated surrogate of the bound in Theorem 1, and the optimality results of Section IV hold for this instantiated problem.

V. Performance Evaluation

In this section, we first compare the proposed MESFL framework with existing distributed learning schemes in terms of latency and test accuracy. We then evaluate the scalability and decision-module overhead, non-independent and identically distributed (non-IID) data partitions, and the sensitivity to the estimated bound constants. We next conduct ablation studies on the impact of the offloading and layer splitting decisions, and finally evaluate MESFL under dynamic environments.

A. Experimental Setup and Reproducibility Protocol

Unless stated otherwise, we consider $N = 20$ devices and $M = 3$ edge servers. The computation capabilities of the devices $\{f_n\}_{n\in\mathcal{N}}$ and edge servers $\{f_m\}_{m\in\mathcal{M}}$ are drawn

[4]The communication cost is $\mathcal{O}(NL)$, which is negligible compared with the activation transmission in one round. The robustness of the decisions to estimation errors is quantified in Section V-E.

uniformly from $[1, 2]$ GHz and $[50, 100]$ GHz, respectively. Uplink transmission rates from devices to edge servers and to the fed server are drawn uniformly from $[60, 100]$ Mbps. All downlink rates are set to 370 Mbps, reflecting the typical uplink–downlink asymmetry in wireless edge systems. We adopt an effective-rate abstraction as in [19], where the rates in Section III-B are instantiated as the above constants. The mini-batch size and learning rate for all schemes are set to 32 and 0.05, respectively. We evaluate the performance on the MNIST [26], CIFAR-10 [27], and CIFAR-100 [27] datasets.

All training latencies reported in this section are model-based. The test accuracy in each round is obtained by training the models, while the corresponding time is computed using the latency model in Section III-B based on the aforementioned capabilities and rates. The latencies of all the baselines are computed from the same latency model with the configuration of each scheme. SplitFed, AdaptSFL, EPSL, and ESFL use a single server with computation capability $\sum_{m\in\mathcal{M}} f_m$ and a common split layer, where AdaptSFL selects its split layer and aggregation interval and ESFL selects its split layer according to their own latency objectives. HSFL keeps the M edge servers with a fixed common split layer and a fixed grouping, with edge aggregation in every round and fed aggregation every E rounds. Vanilla FL trains the full model on the devices and exchanges it with the fed server for aggregation, without any activation exchange. Vanilla SL trains the devices sequentially, so its per-round latency is the sum over the devices and includes the relay of the device-side sub-model between consecutive devices.

We adopt the following reproducibility protocol. Data are partitioned IID, each device receiving $|\mathcal{D}_n| = |\mathcal{D}|/N$ samples drawn uniformly without replacement. Non-IID partitions are described in Section V-D. Unless stated otherwise, results are averaged over five random seeds (42–46) and reported as mean $\pm$ one standard deviation with shaded bands. The memory capacities of the devices and edge servers are drawn from $\mathcal{U}[80, 160]$ MB and $\mathcal{U}[2, 4]$ GB, respectively, with exact draws recorded alongside each seed. The MILPs in Algorithm 2 are solved with Gurobi [24] on a single CPU thread. No time limit is imposed during the training experiments. The fixed-interval subproblems in Section V-C use a 60 s time limit and a 1% relative gap target. For ResNet-18, the implementation is partitioned at eight executable module boundaries: after the stem convolution, stem pooling, residual stages 1–4, average pooling, and flattening. Together with the output layer, the model has $L = 9$ modules and $|\mathcal{L}| = L - 1 = 8$ candidate split points. At each split point, we measure the per-sample FP and BP workloads, the activation size, the device-side sub-model size, and the device-side memory footprint, and these measured values instantiate the latency model of Section III-B. Here, ρ_l counts the multiply-accumulate operations per sample of the convolutional and fully connected layers through layer l, with one operation modeled as one CPU cycle, and $\alpha_l = 2\rho_l$. The sizes $\psi_l = \lambda_l$ and δ_l assume single precision.

We set $T = 120$ for MNIST and $T = 200$ for CIFAR-10 and CIFAR-100, and the aggregation interval is searched over $\mathcal{E} = \{1, \ldots, 10\}$. The latency budget $T^{\max}$ is set to 1.2 times the latency of T rounds under the reference configuration (which features the deepest split layer, round-robin assignment, and $E = 1$). This ensures that fed server aggregation in every round is affordable and MESFL does not synchronize less frequently than the baselines. The fixed-interval subproblems are solved to optimality, and the layer splitting and offloading decisions follow the latency criterion of Section IV-A. Tighter budgets are examined in Section V-E.

TABLE II
LATENCY COMPARISON FOR DISTRIBUTED LEARNING SCHEMES TO REACH TEST ACCURACIES OF 0.9 ON MNIST, 0.7 ON CIFAR-10, AND 0.5 ON CIFAR-100. RESNET-18 WITH $(N, M) = (20, 3)$. THE RESULTS REPRESENT THE MEAN OVER FIVE SEEDS, WITH THE STANDARD DEVIATIONS FOR CIFAR-10 REPORTED IN THE $(20, 3)$ COLUMN OF TABLE IV.

Scheme	**MNIST** ($\times 10^3$ s)	**CIFAR-10** ($\times 10^4$ s)	**CIFAR-100** ($\times 10^4$ s)
Vanilla FL [2]	2.04	29.12	35.42
Vanilla SL [7]	2.46	21.58	27.85
SplitFed [9]	1.24	7.01	15.93
AdaptSFL [19]	0.52	3.02	9.85
HSFL [17]	0.99	5.49	14.06
EPSL [15]	0.83	4.50	12.83
ESFL [18]	0.68	3.80	11.31
MESFL	**0.33**	**1.36**	**4.91**

B. Performance Comparison

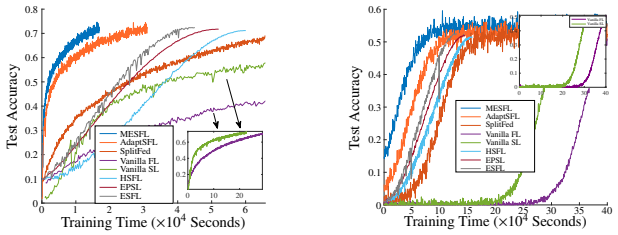
To compare our proposed framework with existing distributed learning schemes, we consider the latency comparison for each scheme to obtain the test accuracy of 0.9 for the MNIST dataset, 0.7 for the CIFAR-10 dataset, and 0.5 for the CIFAR-100 dataset. The baselines include the vanilla federated learning [2], vanilla split learning [7], and five state-of-the-art SFL and PSL schemes, SplitFed [9], AdaptSFL [19], HSFL [17], EPSL [15], and ESFL [18]. For a fair comparison, the native single-server baselines use one edge server with computation capability $\sum_{m\in\mathcal{M}} f_m$, i.e., the same aggregate computation capability as the $M = 3$ edge servers in our framework. HSFL retains its client–edge–cloud hierarchy while its total non-device computation budget is constrained to the same $\sum_{m\in\mathcal{M}} f_m$. To implement the full ML model locally for vanilla FL [2], we do not consider the memory capacity constraint for the devices. Our proposed framework and the baseline schemes share the same initial model $\boldsymbol{\omega}^0$.

Table II and Fig. 3 report the time to reach the target accuracy, and MESFL is the fastest scheme on all three datasets. The gain comes from the grouping induced by the offloading decisions, so that each edge server waits only for its own group rather than for all N devices, together with the split layers and the aggregation interval chosen jointly with that grouping.

We repeat the comparison on VGG-11 and MobileNetV2. Their device-side activation and memory profiles are different from those of ResNet-18. Table III reports the latency to reach 70% CIFAR-10 test accuracy at $(N, M) = (40, 5)$. MESFL retains its advantage across all three architectures.

C. Scalability and Decision-Module Overhead

We next evaluate how the training performance and the overhead of the decision module scale with the system size.



(a) CIFAR-10 (b) CIFAR-100

Fig. 3. Comparison of test accuracy using a ResNet-18 network of the proposed MESFL framework and the vanilla FL, vanilla SL, SplitFed, AdaptSFL, HSFL, EPSL, and ESFL baselines. We train all schemes for the same number of rounds and plot the model-based training time. The different ending times reflect the different per-round latencies of the schemes rather than early stopping.

TABLE III
LATENCY TO REACH 70% CIFAR-10 TEST ACCURACY ($\times 10^4$ S) ACROSS MODEL ARCHITECTURES. $(N, M) = (40, 5)$, RESNET-18 SPLIT AT THE EIGHT EXECUTABLE MODULE BOUNDARIES OF SECTION V-A.

Scheme	ResNet-18	VGG-11	MobileNetV2
SplitFed [9]	7.62 ± 0.94	9.31 ± 0.86	5.47 ± 0.69
AdaptSFL [19]	3.56 ± 0.51	4.46 ± 0.50	2.66 ± 0.24
HSFL [17]	6.07 ± 0.78	7.46 ± 0.72	4.40 ± 0.52
MESFL	$\mathbf{2.11 \pm 0.25}$	$\mathbf{2.68 \pm 0.25}$	$\mathbf{1.71 \pm 0.26}$

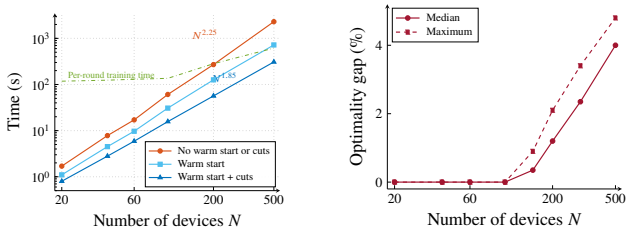
TABLE IV
LATENCY TO REACH 70% CIFAR-10 TEST ACCURACY ($\times 10^4$ S) VERSUS SYSTEM SIZE. RESNET-18.

Scheme	$(20, 3)$	$(40, 5)$	$(60, 5)$	$(100, 8)$
Vanilla FL [2]	29.12 ± 2.47	29.94 ± 3.36	31.08 ± 2.58	32.41 ± 2.81
Vanilla SL [7]	21.58 ± 1.82	22.08 ± 1.75	23.17 ± 0.91	24.36 ± 2.18
SplitFed [9]	7.01 ± 0.54	7.62 ± 0.94	8.86 ± 0.96	9.64 ± 0.85
AdaptSFL [19]	3.02 ± 0.23	3.56 ± 0.51	4.17 ± 0.57	4.55 ± 0.62
HSFL [17]	5.49 ± 0.42	6.07 ± 0.78	7.07 ± 0.81	7.70 ± 0.76
EPSL [15]	4.50 ± 0.34	5.07 ± 0.67	5.91 ± 0.71	6.44 ± 0.71
ESFL [18]	3.80 ± 0.29	4.35 ± 0.59	5.09 ± 0.65	5.55 ± 0.66
MESFL	$\mathbf{1.36 \pm 0.10}$	$\mathbf{2.11 \pm 0.25}$	$\mathbf{2.57 \pm 0.33}$	$\mathbf{2.75 \pm 0.64}$

1) Training Performance at Larger Scale: We repeat the CIFAR-10 comparison with four different system sizes $(N, M) \in \{(20, 3), (40, 5), (60, 5), (100, 8)\}$, keeping the per-entity capability distributions of Section V-A fixed. Table IV reports the time to reach 70% test accuracy. All schemes preserve their relative order from Table II at the four system sizes: vanilla FL and vanilla SL remain roughly an order of magnitude slower than MESFL, while MESFL retains a 38.4%–55.0% latency advantage over the strongest baseline, AdaptSFL. For MESFL, the latency increases by 21.8% from $(40, 5)$ to $(60, 5)$, as the average group size grows from eight to twelve devices, and by only 7.0% from $(60, 5)$ to $(100, 8)$, because the ratio of devices to edge servers remains nearly unchanged.

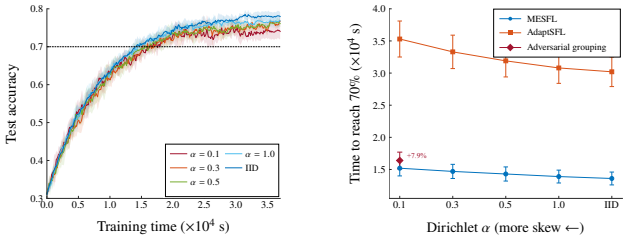
2) Decision-Module Overhead: Fig. 4 reports the wall-clock time of Algorithm 2 to solve problem $\mathbf{P}_3$ (all fixed-interval MILPs, including the warm starts and cuts of Section IV-C). Across the four system sizes of Table IV, the problem solving time grows from 0.8 s at $(N, M) = (20, 3)$ to 15.7 s at $(100, 8)$ on a single CPU thread, and the number of generated feasibility cuts grows from 24 to 52.

Fig. 4(a) varies the number of devices over $N \in \{20, \ldots, 500\}$ on the log–log axes. Over this range, least-



(a) (b)

Fig. 4. Scaling of the decision module (Algorithm 2). Each marker is the median over five random problem instances. (a) Problem solving time versus the number of devices on log–log axes. The annotations show the slopes fitted over the evaluated range, and the dash-dotted curve shows the model-based per-round training time. (b) Anytime optimality gap when each fixed-interval subproblem is limited to 60 s of solver time, combining the per-interval solver gaps with the pruning bound of Lemma 1. The dashed curve shows the maximum gap across the five instances.



(a) (b)

Fig. 5. MESFL under Dirichlet label skew (CIFAR-10, ResNet-18, $(N, M) = (20, 3)$, mean over five seeds). (a) Test accuracy versus model-based training time for $\alpha \in \{0.1, 0.3, 0.5, 1.0\}$ and the IID case. (b) Time to reach the 70% target versus α, for MESFL, AdaptSFL, and the adversarial-grouping arm at $\alpha = 0.1$.

squares fits on the log–log axes give slopes of 1.85 for the complete algorithm and 2.25 for the MILP without warm starts or cuts. These slopes are observed trends over the evaluated range of instances. Within this range, the reduction in problem solving time due to the warm starts and cuts in Section IV-C increases with N.

The dash-dotted curve in Fig. 4(a) is the per-round training time at each evaluated system size, computed from the latency model of Section III-B. The problem solving time of the complete algorithm stays below the per-round training time over the entire evaluated range. This corresponds to an 11.5% fraction of one training round at $(N, M) = (100, 8)$.

Varying M, L, or E^{th} at $N = 20$ changes the problem solving time by less than an order of magnitude, whereas increasing N from 20 to 500 increases the problem solving time by approximately three orders of magnitude, so the number of devices N is the dominant factor in the problem solving time. In Fig. 4(b), the anytime optimality gap from (87) under a solver time limit of 60 s per fixed-interval subproblem is zero up to $N = 100$ and grows to 4.0% at $N = 500$.

D. Non-IID Evaluation

We evaluate MESFL under label-skewed data using the Dirichlet partition that is standard in the FL literature [4]. For each class c, we draw a proportion vector $\boldsymbol{p}_c \sim \text{Dir}(\alpha \mathbf{1}_N)$ and assign the class-c samples to N devices according to $\boldsymbol{p}_c$, fol-

TABLE V
NON-IID SENSITIVITY ON CIFAR-10 AT $(N, M) = (20, 3)$. LATENCY IS THE TIME TO REACH 70% ACCURACY ($\times 10^4$ S), REPORTED AS MEAN $\pm$ STANDARD DEVIATION OVER FIVE SEEDS.

Partition	MESFL	AdaptSFL	$\widehat{\zeta}^2$
IID	1.36 ± 0.10	3.02 ± 0.23	213.4
$\alpha = 1.0$	1.39 ± 0.10	3.08 ± 0.34	234.7
$\alpha = 0.5$	1.43 ± 0.16	3.19 ± 0.31	249.5
$\alpha = 0.3$	1.47 ± 0.11	3.43 ± 0.36	256.3
$\alpha = 0.1$	1.52 ± 0.12	3.53 ± 0.28	278.7
$\alpha = 0.1$, adversarial grouping	1.64 ± 0.13	–	278.7

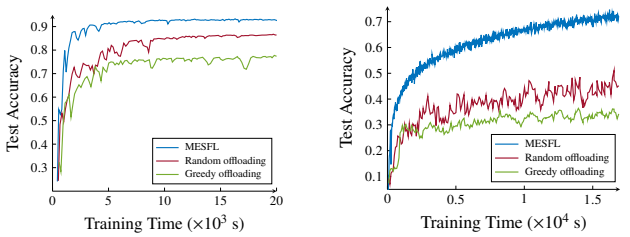
lowed by an equal-size rebalancing step so that $|\mathcal{D}_n| = |\mathcal{D}|/N$ holds for all devices. We consider $\alpha \in \{0.1, 0.3, 0.5, 1.0\}$, where a smaller α induces a stronger skew, together with the IID case. In addition, we run an adversarial-grouping arm at $\alpha = 0.1$, in which the offloading assignment is fixed so that devices with the most similar label distributions are concentrated on the same edge server.

Fig. 5(a) shows the test accuracy versus the model-based training time under each partition. The accuracy curves remain close to the IID curve for all values of α, and a stronger skew mainly delays the time at which the target accuracy is reached. Fig. 5(b) shows that MESFL remains faster than the AdaptSFL baseline under skew, degrading by 11.8% in time-to-target from IID to $\alpha = 0.1$ against 16.9% for AdaptSFL. As the skew sharpens, the measured dissimilarity estimate $\widehat{\zeta}^2$ in Section IV-D grows monotonically from 213.4 under IID to 278.7 at $\alpha = 0.1$ (see Table V), which enters the surrogate (66) through the fourth term $48S^2\eta^2(E-1)^2\zeta^2$ of (23). The selected interval under tighter latency budgets is examined in Section V-E. The adversarial-grouping arm further degrades the time-to-target by 7.9% relative to the optimized assignment at $\alpha = 0.1$ (Fig. 5(b)). This result shows that the grouping induced by the offloading decisions also affects the training performance under concentrated skew.

E. Sensitivity Analysis

We first examine how the latency budget shapes the decisions. Let $T_0^{\max}$ denote the budget of Section V-A. We solve problem $\mathbf{P}_3$ for CIFAR-10 at $(N, M) = (20, 3)$ with $T^{\max}/T_0^{\max} \in \{0.8, 0.6, 0.4\}$ under the IID and the four Dirichlet partitions of Section V-D, using five seeds each. At $T^{\max}/T_0^{\max} = 0.8$, fed server aggregation in every round remains feasible and $E^\star = 1$. At $T^{\max}/T_0^{\max} = 0.6$, $\mathbf{P}_4(1)$ is infeasible, and Algorithm 2 selects $E^\star = 2$ under all five partitions. At $T^{\max}/T_0^{\max} = 0.4$, the selected interval increases further to $E^\star = 4$ for three seeds and $E^\star = 3$ for the other two, again under all five partitions. At these budgets, the selected interval coincides with the smallest feasible one, since the quadratic growth of the drift terms in $E-1$ outweighs the reduction of $\Theta(\boldsymbol{\mu}, \boldsymbol{\beta})$ that a longer interval permits.

We then quantify the robustness of the decisions to errors in the estimated constants at $T^{\max}/T_0^{\max} = 0.6$ by scaling each of $\widehat{H}_l$, $\widehat{\Sigma}_L$, and $\widehat{\zeta}^2$ by $\varkappa \in \{0.5, 0.75, 1.0, 1.5, 2.0\}$, one at a time, and solving problem $\mathbf{P}_3$ again. Across all perturbations and seeds, the selected interval remains $E^\star = 2$, and the relative surrogate increase $[J(\boldsymbol{x}_\varkappa) - J(\boldsymbol{x}^\star)]/J(\boldsymbol{x}^\star)$ under the nominal constants is zero, i.e., every perturbed solution remains optimal under the nominal constants and differs from the nominal solution only among decisions with the same surrogate value.



(a) MNIST (b) CIFAR-10

Fig. 6. Test accuracy using a ResNet-18 network versus training time under different offloading decisions.

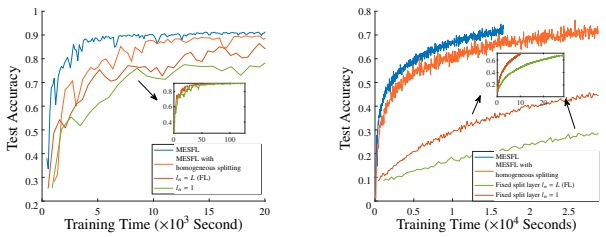
F. Ablation Study

In this subsection, we conduct ablation experiments to illustrate the importance of optimizing the devices' offloading and layer splitting decisions.

1) Different Offloading Decisions: We compare the optimized offloading decisions in our framework, with two baselines: greedy offloading, where all devices choose the (potentially nearer) edge server according to the transmission rates between them, and random offloading. The layer splitting and federated aggregation frequency are optimized. We apply the following admission rule: if the total memory required for an edge server exceeds its capacity, then the server will randomly select a subset of devices to serve in that round and defer the rest. The deferred devices do not perform offloading (i.e., they execute only their device-side layers) and may retry in the next round. Fig. 6 shows that the baselines provide a slower and worse convergence, mainly due to unbalanced load at the edge servers: greedy and random offloading can overload certain servers, so some devices are deferred by the memory limits and participate less in training.

2) Layer Splitting Decisions: In Fig. 7, we compare the training performance under different layer splitting decisions: $l_n = 1$ and $l_n = L$ for all devices, and optimized homogeneous layer splitting with equal $\mu_{n,l}$ for all devices $n \in \mathcal{N}$, where $l_n = L$ (the entire model on the device) is included for comparison although it lies outside the allowable split set in Section V-A. For the fixed layer splitting configurations, we relax the memory capacity constraints for all entities in the baselines to facilitate the comparison. These fixed layer splitting settings result in higher communication overhead and slower convergence.

3) Decoupling Multi-Edge and Joint-Optimization Gains: The comparisons above conflate two sources of improvement: the architectural gain of using multiple edge servers and the algorithmic gain of jointly optimizing $(\boldsymbol{\mu}, \boldsymbol{\beta}, E)$. To decouple them, we compare four configurations on CIFAR-10, which combine one of two architectures with one of two decision policies. The single-edge architecture uses one edge server and its computation capability is equal to $\sum_{m \in \mathcal{M}} f_m$, and the multi-edge architecture uses $M = 3$ edge servers with



(a) MNIST (b) CIFAR-10

Fig. 7. Test accuracy using a ResNet-18 network versus training time under different layer splitting decisions.

TABLE VI
DECOUPLING THE ARCHITECTURAL AND ALGORITHMIC GAINS: LATENCY TO REACH 70% CIFAR-10 TEST ACCURACY ($\times 10^4$ S). RESNET-18, $N = 20$, THE SAME TOTAL SERVER-SIDE COMPUTATION CAPABILITY IN ALL CONFIGURATIONS. MEAN $\pm$ STD OVER FIVE SEEDS.

	Fixed split + random offload	Joint optimization
Single edge (M=1)	4.20 ± 0.36	2.85 ± 0.25
Multi-edge (M=3)	2.95 ± 0.23	$\mathbf{1.36 \pm 0.10}$ (MESFL)

individual capabilities f_m, so both architectures have the same total server-side capability. The fixed policy uses a fixed split layer with random offloading, and the optimized policy uses Algorithm 2. Table VI reports the time to reach 70% test accuracy for the four configurations. Comparing the two rows isolates the gain of the multi-edge architecture (29.8% under the fixed policy and 52.3% under the optimized policy), and comparing the two columns isolates the gain of the joint optimization (32.1% with a single edge server and 53.9% with multiple edge servers). The upper-left entry is the single-edge, fixed-policy configuration, which is the closest analogue of SplitFed among the four entries, and only the lower-right entry, which is MESFL, realizes both gains. The two gains are therefore complementary rather than redundant.

G. Dynamic Environments

The analysis in Section III and the optimization in Section IV treat the computation capabilities and link rates as fixed. In practice, the computation capabilities vary with thermal throttling and background load, and the link rates vary with the channel conditions. We therefore interpret the decisions $(\boldsymbol{\mu}, \boldsymbol{\beta}, E)$ as quasi-static epoch decisions: each epoch runs a fixed configuration of the form analyzed by Theorem 1, and the decisions are re-optimized when the measured capabilities drift. Since the theorem does not provide an end-to-end guarantee across configuration changes, the study in this subsection is empirical.

We generate time-varying capabilities $f_n(t) = \bar{f}_n\big(1 + \nu_n(t)\big)$ and similarly perturbed uplink rates $r^{\mathrm{UL}}_{n,m}(t)$, where each perturbation follows an independent stationary first-order autoregressive (AR(1)) process with memory parameter $\varphi = 0.98$ and relative standard deviation $\sigma \in \{0.15, 0.30\}$, clipped to $\pm 50\%$. All policies below are evaluated on paired traces (common random realizations) over five seeds.

We compare three ways of operating MESFL: (i) nominal-static, which solves problem $\mathbf{P}_3$ once with the long-run mean capabilities and holds the decisions fixed, (ii) conservative-static, which solves once with the 10th-percentile capabilities as a conservative provisioning strategy that requires no tracking, and (iii) re-optimization, which solves problem $\mathbf{P}_3$ again every W_{re} rounds using the trailing-window means of the measured capabilities and rates, with $W_{\mathrm{re}} \in \{20, 50\}$.

TABLE VII
LATENCY TO REACH 70% CIFAR-10 TEST ACCURACY ($\times 10^4$ S) UNDER AR(1) TIME-VARYING COMPUTATION CAPABILITIES AND LINK RATES ($\varphi = 0.98$, PAIRED TRACES). RESNET-18, $(N, M) = (20, 3)$.

Policy	Static (σ=0)	σ=0.15	σ=0.30
AdaptSFL [19]	3.02 ± 0.23	3.20 ± 0.29	3.46 ± 0.38
MESFL, nominal-static	1.36 ± 0.10	1.47 ± 0.14	1.61 ± 0.20
MESFL, conservative-static	1.36 ± 0.10	1.43 ± 0.13	1.50 ± 0.17
MESFL, re-opt. W_{re}=50	1.36 ± 0.10	1.41 ± 0.12	1.47 ± 0.16
MESFL, re-opt. W_{re}=20	1.36 ± 0.10	$\mathbf{1.39 \pm 0.11}$	$\mathbf{1.43 \pm 0.14}$

Table VII reports the time to reach 70% CIFAR-10 test accuracy under each policy. At $\sigma = 0.30$, the nominal-static policy loses 18.4% relative to the static case, conservative provisioning recovers 44.0% of this loss without any tracking, and re-optimization recovers 56.0% with $W_{\mathrm{re}} = 50$ and 72.0% with $W_{\mathrm{re}} = 20$. At $\sigma = 0.15$, the loss shrinks to 8.1%, and re-optimization with $W_{\mathrm{re}} = 20$ again recovers 72.7% of it, so the recovery fraction is insensitive to the perturbation strength. Each re-optimization costs 0.8 s at $(N, M) = (20, 3)$, below 0.1% of the training time of one re-optimization window. Notably, MESFL under the strongest perturbation and without any tracking remains faster than AdaptSFL under static conditions (1.61 versus 3.02 $\times 10^4$ s), so the advantage of the framework does not rest on tracking the time-varying resources.

VI. CONCLUSION

In this paper, we proposed MESFL, which is a generalized framework for federated split learning when the devices and edge servers have different memory and computational capabilities. The combination of edge and fed servers ensures a balance between computational efficiency and model convergence, whereas heterogeneous layer splitting enables devices to optimize resource utilization based on their individual capabilities. By jointly optimizing the sub-model offloading decisions, layer splitting, and federated aggregation frequencies, we minimized the learning convergence bound while satisfying the latency and memory constraints. By exploiting the bounded aggregation interval, we decomposed the bound-based MINLP exactly into a finite family of fixed-interval MILPs. The evaluation compares MESFL with baseline schemes in terms of test accuracy and the latency to reach a target accuracy. For future work, we will explore adaptive federated aggregation frequencies among different groups, which introduces new challenges in mitigating bias in the learning convergence analysis.

REFERENCES

[1] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proc. of the IEEE*, vol. 107, no. 8, pp. 1738–1762, Aug. 2019.

[2] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Int'l Conf. on Artificial Intelligence and Statistics*, Fort Lauderdale, FL, Apr. 2017.

[3] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, Mar. 2021.

[4] S. Wang, J. Perazzone, M. Ji, and K. S. Chan, "Federated learning with flexible control," in *Proc. IEEE Int'l Conf. Comp. Commun. (INFOCOM)*, New York, NY, May 2023.

[5] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, Sep. 2014.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. of IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, Jun. 2016.

[7] P. Vepakomma, O. Gupta, T. Swedish, and R. Raskar, "Split learning for health: Distributed deep learning without sharing raw patient data," in *Proc. Int'l Conf. Learning Representations (ICLR) Workshop on AI for Social Good*, New Orleans, LA, May 2019.

[8] Z. Lin, G. Qu, X. Chen, and K. Huang, "Split learning in 6G edge networks," *IEEE Wireless Commun.*, vol. 31, no. 4, pp. 170–176, Aug. 2024.

[9] C. Thapa, P. C. M. Arachchige, S. Camtepe, and L. Sun, "SplitFed: When federated learning meets split learning," in *Proc. of AAAI Conf. on Artificial Intelligence*, Jun. 2022.

[10] V. Turina, Z. Zhang, F. Esposito, and I. Matta, "Federated or split? A performance and privacy analysis of hybrid split and federated learning architectures," in *Proc. Int'l Conf. on Cloud Comput. (CLOUD)*, Sep. 2021, pp. 250–260.

[11] X. Liu, Y. Deng, and T. Mahmoodi, "Wireless distributed learning: A new hybrid split and federated learning approach," *IEEE Trans. Wirel. Commun.*, vol. 22, no. 4, pp. 2650–2665, Apr. 2023.

[12] W. Wu, M. Li, K. Qu, C. Zhou, X. Shen, W. Zhuang, X. Li, and W. Shi, "Split learning over wireless networks: Parallel design and resource management," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 4, pp. 1051–1066, Apr. 2023.

[13] S. Nair, M. Lin, P. Ju, A. Talebi, E. S. Bentley, and J. Liu, "FSL-SAGE: Accelerating federated split learning via smashed activation gradient estimation," in *Proc. Int'l Conf. Machine Learning (ICML)*, Vancouver, Canada, Jul. 2025.

[14] P. Joshi, C. Thapa, S. Camtepe, M. Hasanuzzamana, T. Scully, and H. Afli, "Splitfed learning without client-side synchronization: Analyzing client-side split network portion size to overall performance," in *Proc. Collaborative European Research Conf. (CERC)*, Cork, Ireland, Sep. 2021.

[15] Z. Lin, G. Zhu, Y. Deng, X. Chen, Y. Gao, K. Huang, and Y. Fang, "Efficient parallel split learning over resource-constrained wireless edge networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9224–9239, Oct. 2024.

[16] C. Xu, J. Li, Y. Liu, Y. Ling, and M. Wen, "Accelerating split federated learning over wireless communication networks," *IEEE Trans. Wirel. Commun.*, vol. 23, no. 6, pp. 5587–5599, Jun. 2024.

[17] Z. Lin, W. Wei, Z. Chen, C.-T. Lam, X. Chen, Y. Gao, and J. Luo, "Hierarchical split federated learning: Convergence analysis and system optimization," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 9352–9367, Oct. 2025.

[18] G. Zhu, Y. Deng, X. Chen, H. Zhang, Y. Fang, and T. F. Wong, "ESFL: Efficient split federated learning over resource-constrained heterogeneous wireless devices," *IEEE Internet of Things J.*, vol. 11, no. 16, pp. 27 153–27 166, 2024.

[19] Z. Lin, G. Qu, W. Wei, X. Chen, and K. K. Leung, "AdaptSFL: Adaptive split federated learning in resource-constrained edge networks," *IEEE Trans. Networking*, vol. 33, no. 6, pp. 2993–3008, Dec. 2025.

[20] T. Kim, J. Zuo, X. Zhang, and C. Joe-Wong, "Edge-MSL: Split learning on the mobile edge via multi-armed bandits," in *Proc. IEEE Int'l Conf. Comp. Commun. (INFOCOM)*, Vancouver, Canada, May 2024.

[21] J. Tirana, D. Tsigkari, G. Iosifidis, and D. Chatzopoulos, "Workflow optimization for parallel split learning," in *Proc. IEEE Int'l Conf. Comp. Commun. (INFOCOM)*, Vancouver, Canada, May 2024.

[22] Y. Sun, J. Xu, and S. Cui, "User association and resource allocation for MEC-enabled IoT networks," *IEEE Trans. Wirel. Commun.*, vol. 21, no. 10, pp. 8051–8062, Oct. 2022.

[23] D.-J. Han, D.-Y. Kim, M. Choi, D. Nickel, J. Moon, M. Chiang, and C. G. Brinton, "Federated split learning with joint personalization-generalization for inference-stage optimization in wireless edge networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 6, pp. 7048–7065, Jun. 2024.

[24] "Gurobi solver," https://www.gurobi.com/, accessed: July 1, 2026.

[25] L. A. Wolsey, *Integer Programming*, 2nd ed. Hoboken, NJ: Wiley, 2020.

[26] Y. LeCun, C. Cortes, and C. J. Burges, "The MNIST database of handwritten digits," http://yann.lecun.com/exdb/mnist/, accessed: July 1, 2026.

[27] A. Krizhevsky, V. Nair, and G. Hinton, "The CIFAR-10 dataset," https://www.cs.toronto.edu/∼kriz/cifar.html, accessed: July 1, 2026.



Shu Hong is a Postdoctoral Associate in the Department of Electrical and Computer Engineering at George Washington University. She received the Ph.D. degree from Singapore University of Technology and Design, in 2023. She was a postdoctoral fellow at the University of British Columbia and a visiting student at Purdue University. Her research interests include trustworthy and efficient learning in edge, networked, and cyber-physical systems. She has received the N2Women Young Researcher Fellowship, the Best Poster/Demo Award at the ACM MobiHoc XR Security Workshop, the President's Graduate Fellowship, SUTD Outstanding Thesis Award, and multiple travel grants.



Vincent W.S. Wong (S'94, M'00, SM'07, F'16) received the B.Sc. degree from the University of Manitoba, Canada, in 1994, the M.A.Sc. degree from the University of Waterloo, Canada, in 1996, and the Ph.D. degree from the University of British Columbia (UBC), Vancouver, Canada, in 2000. From 2000 to 2001, he worked as a systems engineer at PMC-Sierra Inc. (now Microchip Technology Inc.). He joined the Department of Electrical and Computer Engineering at UBC in 2002 and is currently a Professor and UBC Distinguished University Scholar. His research areas include protocol design, optimization, and resource management of communication networks, with applications to 5G/6G wireless networks, Internet of things, mobile edge computing, smart grid, and energy systems. Dr. Wong is the Editor-in-Chief of the *IEEE Transactions on Wireless Communications*. He has served as an Area Editor of the *IEEE Transactions on Communications* and *IEEE Open Journal of the Communications Society*, an Associate Editor of the *IEEE Transactions on Mobile Computing* and *IEEE Transactions on Vehicular Technology*, and a Guest Editor of the *IEEE Journal on Selected Areas in Communications*, *IEEE Internet of Things Journal*, and *IEEE Wireless Communications*. Dr. Wong is the TPC Vice-Chair of *IEEE GLOBECOM* 2027, Publicity Co-Chair of *IEEE VTC2027-Spring*, and Area Chair of *IEEE INFOCOM* 2027. He has served as the General Co-Chair of *IEEE INFOCOM* 2024; Tutorial Co-Chair of *IEEE GLOBECOM* 2018; Technical Program Co-Chair of *IEEE VTC*2020-*Fall* and *IEEE SmartGridComm* 2014; and Symposium Co-Chair of *IEEE ICC*'18, *IEEE SmartGridComm* ('13, '17) and *IEEE GLOBECOM*'13. He received the 2022 Best Paper Award from *IEEE Transactions on Mobile Computing* and Best Paper Awards at the *IEEE ICC* 2022 and *IEEE GLOBECOM* 2020. He has served as the Chair of the IEEE Vancouver Joint Communications Chapter and IEEE Communications Society Emerging Technical Sub-Committee on Smart Grid Communications. He was an IEEE Communications Society Distinguished Lecturer from 2019 to 2020 and is an IEEE Vehicular Technology Society Distinguished Lecturer for the term of 2023−2026. Dr. Wong is a Fellow of the IEEE, Canadian Academy of Engineering, and the Engineering Institute of Canada.